

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное

Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕНА К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ АДМИНИСТРИРОВАНИЯ
ГРУППОВЫХ ЗАНЯТИЙ**

по направлению 09.03.03 Прикладная информатика

Образовательная программа (профиль)

«Корпоративные информационные системы»

Студент: _____ / Сыров Максим Евгеньевич, 211-361/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Васильев Денис Борисович/
подпись *ФИО, уч. звание и степень*

Москва 2025

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное

Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

по направлению 09.03.03 Прикладная информатика

Образовательная программа (профиль)

«Корпоративные информационные системы»

Тема ВКР	Разработка веб-приложения администрирования групповых занятий
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Автоматизация процессов управления и администрирования групповых занятий при проведении образовательной деятельности
Основные функции	1. Создание групп обучающихся и занятий внутри группы с разным количеством участников. 2. Организация учебного процесса путем ведения учета посещаемости, оплаты и успеваемости каждого ученика внутри группы. 3. Просмотр и создание гибкого расписания (разовые, повторяющиеся). 4. Уведомление учащихся о переносе или отмены занятий. 5. Обеспечение информационной безопасности за счет реализации аутентификации с использованием JWT-токенов и защищенных Куки.
Используемые технологии и платформы	1. FastAPI. 2. Vue.js. 3. JavaScript. 4. PostgreSQL. 5. JWT. 6. Git. 7. Docker. 8. HTML. 9. CSS.

Спроектировать и описать бизнес процессы.																	
Спроектировать и разработать систему.																	
Спроектировать пользовательский интерфейс.																	
Провести тестирование функционала																	
Провести развертывание веб приложения																	
Сделать выводы о результатах применения системы и выполненных целях.																	

РУКОВОДИТЕЛЬ ОП:

«__»_____2025_____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»_____2025_____ / Васильев Денис Борисович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»_____2025_____ / Сыров Максим Евгеньевич, 211-361 /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: Разработка веб-приложения для администрирования групповых занятий

Цель работы: Создать интуитивно понятную платформу, которая объединит функции планирования, учета и финансового контроля, адаптированную под потребности преподавателей и частных образовательных учреждений.

Работа состоит из 3 глав, Введения, Заключения и Приложений. Общий объем работы 111 страниц, из них 3 приложений на 31 листах. В работу включены 36 рисунков, 2 таблицы. Библиографический список состоит из 51 источников, из них 10 печатных источников, 41 интернет-источников.

Во Введении описаны цель и задачи работы, объект и предмет исследования, его актуальность, новизна и практическая значимость.

В Первой главе описывается аналитическая часть разработки. Исследована предметная область, целевая аудитория и требования к системе

Во Второй главе представлена разработка серверной и клиентской части веб-приложения.

В Третьей главе раскрыты технические аспекты внедрения и опытной эксплуатации полученного практического результата ВКР.

В Заключении представлены выводы по поставленным задачам.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ.....	9
1.1 Описание предметной области	9
1.2 Анализ целевой аудитории	9
1.3 Анализ аналогов и конкурентов	15
1.4 Анализ требований	18
1.5 Описание необходимого функционала	19
1.6 Выводы по аналитической части	23
2 ПРОЕКТНАЯ ЧАСТЬ	24
2.1 Анализ технических требований	24
2.2 Проектирование базы данных	24
2.3 Проектирование и разработка серверной части приложения	31
2.4 Разработка интерфейса	41
2.5 Проектирование и разработка клиентской части приложения	48
2.6 Выводы по проектной части	61
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ	62
3.1 Развертывание приложения	62
3.2 Проведение тестирования сервиса	67
3.3 Анализ возможностей развития приложения	69
3.4 Выводы по внедрению и эксплуатации	72
ЗАКЛЮЧЕНИЕ	74
СПИСОК ЛИТЕРАТУРЫ	75
ПРИЛОЖЕНИЕ А.....	80
ПРИЛОЖЕНИЕ Б	94
ПРИЛОЖЕНИЕ В.....	97

ВВЕДЕНИЕ

В условиях стремительной цифровизации образования наблюдается устойчивый рост спроса [1] на специализированные сервисы для организации учебного процесса. Особенно остро эта потребность проявляется в сегменте частных школ и индивидуального обучения, где преподаватели сталкиваются с необходимостью ручного управления расписаниями, посещаемостью и платежами. Современные LMS-системы, несмотря на их функциональность, часто оказываются избыточно сложными [2] для решения узкоспециализированных задач группового обучения, что создает рыночную нишу для интуитивно понятных решений.

Большинство доступных инструментов предлагают либо фрагментированный функционал (например, отдельные сервисы для планирования занятий или учета платежей), либо требуют значительных временных затрат на освоение. Преподаватели вынуждены комбинировать несколько платформ, что приводит к потере данных, дублированию операций и снижению эффективности. Критически не хватает комплексного продукта, объединяющего ключевые функции администрирования учебного процесса с фокусом на удобство для малых образовательных проектов.

Разрабатываемый сервис призван заполнить этот пробел за счет интеграции возможностей в единую платформу, такие как автоматизированное формирование расписаний с учетом ресурсов преподавателей, встроенная система учета посещаемости и платежей и создание и модерация объединенных обучающихся в отдельные группы.

Основными пользователями станут частные преподаватели, ведущие групповые занятия, мини-школы и образовательные центры.

Ключевое конкурентное преимущество — специализация на потребностях малого сегмента: от оптимизированного интерфейса до автоматизации типовых сценариев (например, массовые уведомления об изменениях в расписании).

Новизна предполагаемого решения заключается в том, что в отличие от универсальных LMS, платформа предлагает гибкую систему лимитов, адаптированную под размер учебных групп, а также минимизацию рутинных операций за счет шаблонов и массовых действий.

Проект разрабатывается как стартап с потенциалом масштабирования. Пилотное тестирование планируется провести среди локальных образовательных проектов, после чего сервис может быть адаптирован для других рынков (например, корпоративного обучения). Долгосрочная цель — создание экосистемы с API-доступом для интеграции с внешними сервисами (платежные системы, календари).

Объект и предмет исследования: Веб-приложение для администрирования групповых занятий

Задачи для достижения цели:

1. Проанализировать предметную область.
2. Проанализировать целевую аудиторию.
3. Проанализировать конкурентов и аналоги.
4. Определить требования к системе.
5. Спроектировать и описать бизнес процессы.
6. Спроектировать и разработать веб-приложение.
7. Спроектировать пользовательский интерфейс.
8. Протестировать систему.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Описание предметной области

В настоящее время в сфере образования, в частности, в сегменте частных школ и индивидуального обучения, растет запрос к современному подходу в организации учебного процесса. Преподаватели тратят значительное время на рутинные задачи: формирование расписаний, учет посещаемости, контроль оплаты занятий и коммуникацию с учениками [3]. При этом существующие инструменты часто оказываются либо слишком сложными (например, системы управления обучением — LMS), либо недостаточно специализированными под нужды групповых занятий.

Разработка веб-приложения для администрирования групповых занятий направлена на решение этих проблем. Цель проекта — создать интуитивно понятную платформу, которая объединит функции планирования, учета, финансового контроля и коммуникации, адаптированную под потребности преподавателей и частных образовательных учреждений. Приложение позволит автоматизировать ключевые процессы, снизив нагрузку на пользователей и повысив прозрачность учебного процесса для учеников и их родителей.

1.2 Анализ целевой аудитории

Целевая аудитория (ЦА) [4] — это группа потенциальных потребителей какого-то продукта, люди, которые вероятнее всего заинтересуются товаром или услугой.

Для определения портрета целевой аудитории был проведен опрос, направленный на представителей образовательной сферы деятельности. Респондентам было представлено 9 вопросов, по результатам которых можно оценить портрет целевой аудитории, проблемы с которыми она сталкивается, а также понять основной запрос целевой аудитории в рамках образовательного процесса.

В опросе приняли участие 37 респондентов, подавляющее число из которых - представители молодого поколения, в возрасте от 20 до 30 лет (рисунк 1.1).

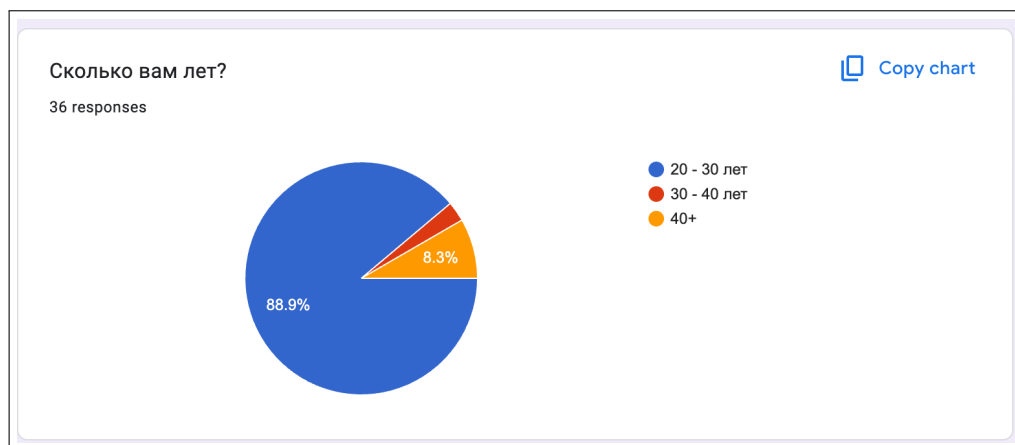


Рисунок 1.1 – Результаты демографии

Касаемо профессии, больше половины принявших участие респондентов являются частными репетиторами. На 2 месте по частоте - преподаватели частных школ (рисунок 1.2).

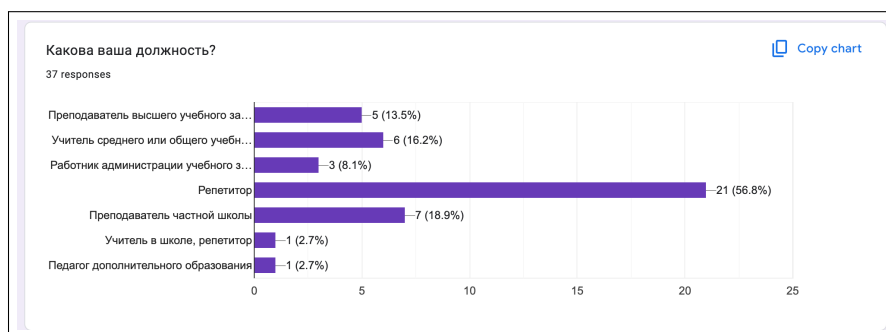


Рисунок 1.2 – Статистика по профессии

Благодаря этой метрике, можно сегментировать ответы и определить в какой сфере относятся самые часто встречающиеся трудности.

Далее, согласно вопросу об опыте в сфере образования (рисунок 1.3), 45.9% опрошенных имеют опыт от 2 до 5 лет, 40.5% только начинают свой путь в образовательной деятельности, и лишь 13% опрошенных имеют более серьезный опыт.

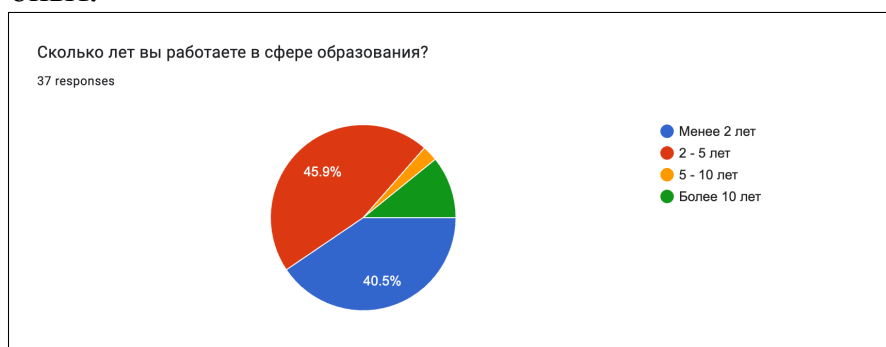


Рисунок 1.3 – Опыт респондентов

Представленные показатели дают основание для более детального изучения факторов, влияющих на эффективность педагогической деятельности. Сравнительный анализ временных затрат различных категорий педагогов открывает новые перспективы для оптимизации рабочего процесса.

Следующий вопрос был направлен на выяснение самых времязатратных задач в работе респондентов (рисунок 1.4).

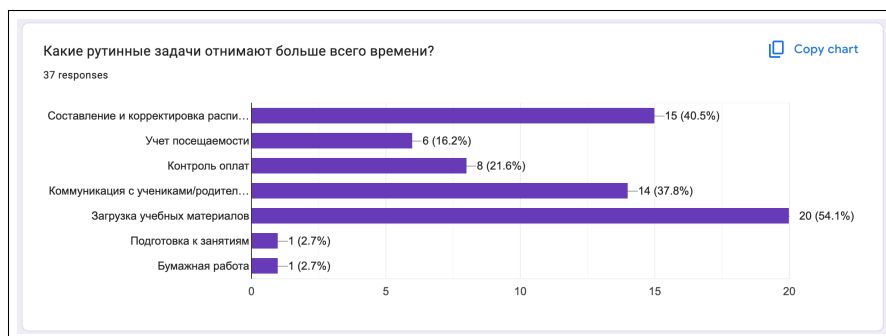


Рисунок 1.4 – Самые времязатратные задачи

Из рисунка 1.4 можно сделать вывод о том, что самой времязатратной задачей является загрузка учебных материалов, составление и корректировка расписания и коммуникация с учениками.

Также из общей таблицы ответом можно получить информацию о том, что с проблемой загрузки учебных материалов сталкиваются в основном репетиторы, когда с проблемой коммуникации сталкиваются в основном учителя среднего образования, а учет посещаемости является главной проблемой преподавателей частных школ. Это говорит о том, что эти процессы требуют автоматизации.

На рисунке 1.5 представлены результаты вопроса, направленного на выявление инструментария, которым пользуются респонденты сейчас для решения своих повседневных задач.



Рисунок 1.5 – Наиболее используемые инструменты

Из ответов видно, что 64% опрошенных предпочитают вести учет своей деятельности на бумаге и лишь 35% предпочитают использовать электронные носители. Это дает сигнал о том, что многие респонденты не готовы использовать специализированные сервисы, поскольку, как показано на рисунке 7, эти же респонденты, что выбрали основной инструмент - бумагу, ответили, что главной их проблемой является дублирование и потеря информации. Исходя из этого можно сделать вывод о том, что эта часть целевой аудитории желает перейти от традиционного ведения учета на бумаге к более подходящему решению и испытывает трудности.

Также, из рисунка 1.5 можно сделать простой вывод о том, что хоть 45% ответили, что используют специализированные сервисы, также используют Excel. Это говорит о фрагментации данных. Респонденты вынуждены использовать множество сервисов одновременно и это подтверждает нишу моего решения.

На рисунке 1.6 представлен вопрос о владения компьютером.



Рисунок 1.6 – Уровень владения компьютером

Из результатов вопроса можно сделать уверенный вывод о том, что большая часть целевой аудитории имеет высокий или же средний уровень владения персонального компьютера. Это значит, что внешний вид и интерфейс веб-приложения должен быть простым, интуитивно понятным и не вызывать большой когнитивной нагрузки. Он должен соответствовать базовым правилам читаемости текста на экранах.

Вопрос 7 (рисунок 1.7) нацелен на выявление уровней удовлетворенности и неудовлетворенности целевой аудитории с сервисами, которые они привыкли использовать сейчас.



Рисунок 1.7 – Удовлетворенность текущими решениями

Здесь видно, что мнения респондентов разделились. Около трети респондентов ответили, что им не хватает некоторого функционала, 16% заявили им не хватает критически важного функционала и испытывают проблемы с хранением информации. Это дает основание полагать, что мое решение будет особенно актуально для минимизирования ручного труда.

По итогам 8 вопроса, представленного на рисунке 1.8, можно сделать вывод о том, что весь перечисленный функционал востребован среди целевой аудитории.

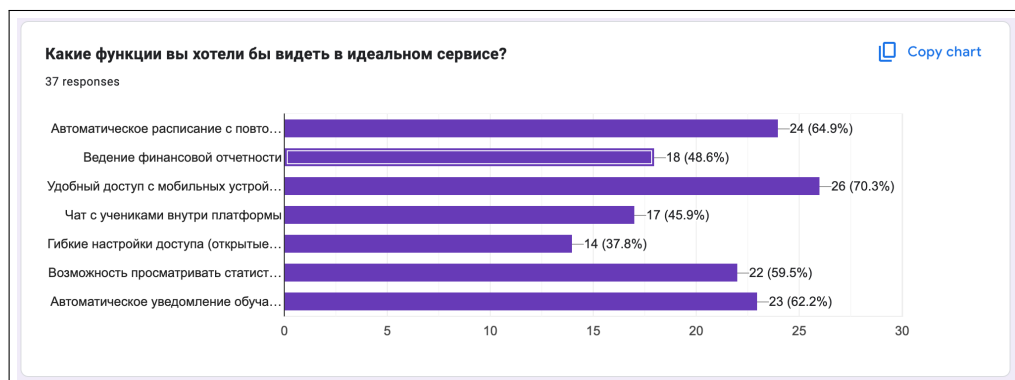


Рисунок 1.8 – Функции идеального сервиса

Из сводной таблицы, можно сделать вывод о том, что те респонденты, которые указали своей главной проблемой "Составление и корректировка расписания", использующие Excel больше всего выбрали желаемый функционал "Автоматическое расписание с повторяемыми занятиями", а также "Ведение финансовой отчетности".

Также, те респонденты, чей уровень владения компьютером является начинающим или средним, заявили о своей потребности в существовании адаптированной версии веб приложения под мобильные устройства, такие как телефон, планшет и небольшие компьютеры.

А гибкие настройки доступа, разделение групп обучающихся на открытые и закрытые, отметили в основном репетиторы и преподаватели частных школ и заведений.

Из результатов проведенного опроса, можно составить портрет среднестатистического пользователя системы администрирования групповых занятий.

Портрет ЦА:

- 1) возраст: 25–35 лет;
- 2) опыт работы: 2-10 лет;
- 3) техническая подготовка: средний уровень. Многие не имеют опыта работы с профессиональными CRM или LMS, но активно используют базовые инструменты (Яндекс Календарь, Excel, мессенджеры).

Потребности:

1. Быстрое создание групп и расписаний без ручного ввода данных.
2. Учет посещаемости с возможностью отметки в реальном времени.
3. Автоматический расчет доходов.
4. Доступ к статистике (посещаемость, оплаты, успеваемость).

Проблемы, с которыми сталкиваются:

1. Фрагментация инструментов: расписание в одном месте, оплаты — в другом, общение — в мессенджере.
2. Сложности с контролем задолженностей учеников.
3. Отсутствие единого пространства для работы с группами.
4. Недостаток коммуникации с обучающимися.

Также, для автоматизации процесса ведения групп, а также расширения функциональности системы, проанализирована вторичная аудитория: обучающиеся.

Портрет ЦА:

- 1) возраст обучающихся: 12–25 лет;
- 2) техническая подготовка: средняя. Хорошо владеют компьютером и телефоном, но не имеют навыков работы в профессиональных CRM системах и LMS.

Исходя из ответов респондентов, представленные в ответах на вопросы, а также используя портрет целевой аудитории, можно определить основные потребности основной целевой аудитории:

- 1) просмотр расписания занятий и домашних заданий;
- 2) запись на групповые или индивидуальные занятия;
- 3) доступ к истории оплат и посещаемости.

Проблемы, с которыми сталкиваются:

- 1) необходимость уточнять расписание у преподавателя;
- 2) отсутствие прозрачности в оплатах.

По сформированным портретам основной целевой аудитории можно определиться с списком функциональных и нефункциональных требований к системе, которые будут нацелены на улучшение пользовательского опыта и решение главных задач целевой аудитории.

1.3 Анализ аналогов и конкурентов

Анализ аналогов [5] – это процесс изучения существующих решений (конкурентов, продуктов, сервисов) с целью выявления их сильных и слабых сторон, функционала, ценовой политики и пользовательского опыта.

Анализ аналогов поможет понять, какие функции и подходы наиболее востребованы пользователями, а также выявить слабые стороны существующих решений.

К сожалению, сегодня большинство преподавателей и репетиторов предпочитают использовать excel таблицы.

Преимущества:

1. Приемственность. Многие используют Excel поскольку десятилетиями использовали его для выполнения всех своих рутинных задач и попросту не хотят переходить на что-то более новое.

2. Доступность. Excel предустановлен по умолчанию на всех Windows устройствах и доступен сразу "из коробки". Пользователю сразу доступен весь основной функционал приложения без необходимости устанавливать дополнительное ПО самому.

3. Удобство хранения. Некоторые xlsx редакторы поддерживают облачное хранение, обеспечивая своим пользователем общий доступ с разных устройств (при подключении их устройств к сети интернет), лишая их необходимости в ношении своего устройства накопления информации на постоянной основе.

Недостатки:

1. Сбор информации. Пользователю необходимо вручную вводить информацию по каждому своему ученику на каждое занятие. На это тратится много времени, а также много теряется на построение коммуникации вне системы.

2. Отсутствие целевой направленности. Изначально excel не строился как платформа для моей целевой аудитории, а следовательно не может учитывать их потребности.

3. Переполненность функционалом. Excel редактор представляет собой многофункциональное приложение, способности которого только затрудняют навигацию по приложению, отвлекая тем функционалом, который не требуется для достижения целей ЦА.

Из опроса, был выявлен неочевидный, но распространенный инструмент - бумажный носитель.

Преимущества:

1. Доступность. Блокнот или ежедневник, как правило, всегда под рукой у преподавателя.

2. Простота использования. Поскольку писать могут абсолютно все, вести учет и заметки можно в любом виде.

Недостатки:

1. Ненадежность. Бумага - материал легко воспламеняемый и подвержен влаге. Его крайне легко испортить, а значит быстро утратить важную информацию.

2. Вреязатратность. Ведение ежедневника обязывает тратить значительное время на организацию и структуризацию данных. Это является главным недостатком, поскольку это время сегодня - главный ресурс.

Ярепетитор [6] - прямой конкурент моей платформе, который предоставляет общий функционал создания занятий, просмотра расписания, учета заработка и оплаченных часов.

Однако, эта платформа обладает главным недостатком - односторонность использования. Система предоставляет функционал только для преподавателя и совсем игнорирует потребности обучающихся, чья ориентированность занимает ключевую роль в проведении образовательной деятельности.

Преимущества:

1. Наличие календаря и возможность вести расписание.
2. Возможность вести статистику заработка.
3. Многоплатформенность. Сервис доступен как в веб версии, так и android и IOS приложение.
4. Наличие файлообменника. Сервис предоставляет возможность хранения заданий и материалов к занятиям.
5. Интеграция с сторонними сервисами. Сервис поддерживает авторизацию через Google и Apple аккаунты.

Недостатки:

1. Пользователю приходится вручную заносить каждого своего ученика в систему, а также всю информацию о нем.
2. Пользователю приходится самому уведомлять своих учениках о переносе занятия.
3. Ученики пользователя не имеют возможности получить общую информацию о курсах и занятиях преподавателя без предварительной коммуникации.

Это порождает лишнее потраченное время на согласование времени, цены и условия проведения занятий.

Также, ЯРепетитор выделяет сложный и неинтуитивный интерфейс, затрудняющий использования расписания и создание занятий (рисунок 1.9).

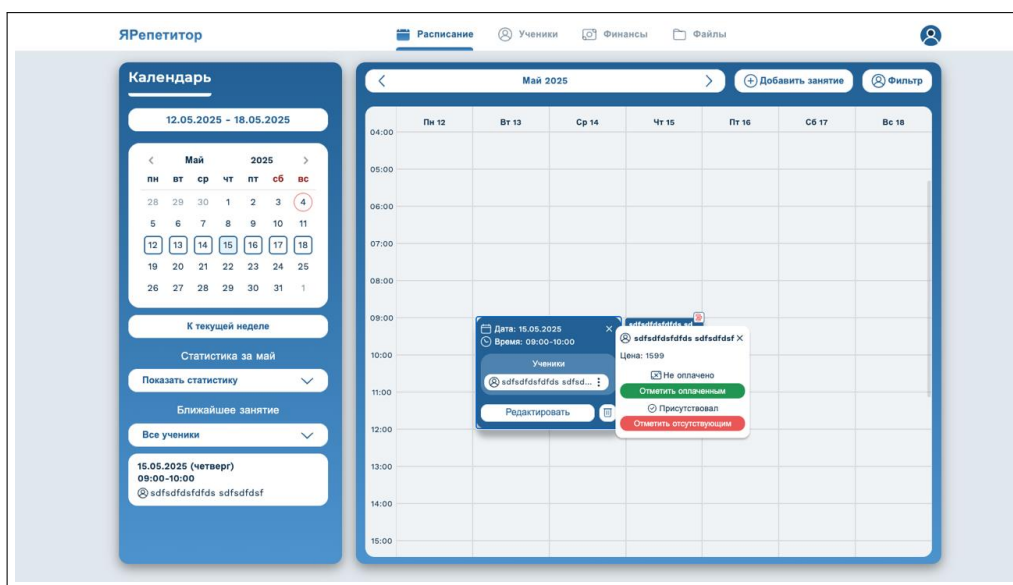


Рисунок 1.9 – Пример интерфейса аналога

Визуальные недостатки аналога:

1. Проблемы с контрастностью и читаемостью. Слишком светлые или неконтрастные элементы интерфейса, например, серые кнопки или текст на белом фоне, что затрудняет восприятие информации, особенно для пользователей с ослабленным зрением.

2. Неудобная навигация и организация контента. Сложная структура меню, из-за которой пользователи вынуждены совершать лишние действия для доступа к ключевым функциям (например, редактирование расписания через несколько шагов).

3. Ошибки в отображении данных. Непоследовательное обновление интерфейса: пользователи отмечают, что после оплаты подписки или добавления учеников изменения не отражаются сразу, что вызывает путаницу 2 (отзывы Nas7usha7, T3lfi).

4. Использование устаревших UI-компонентов, которые выглядят нефункционально (например, плоские кнопки без теней или градиентов).

Преимущества:

1. Специализация на групповых занятиях. Инструменты заточены под нужды преподавателей: быстрое создание групп, массовая отметка посещаемости.

2. Все в одном месте. Объединение расписания, оплат, коммуникации и контента избавляет от необходимости переключаться между сервисами.

3. Простота для не технических пользователей. Минимум настроек, интуитивные формы, подсказки при первом входе.

4. Финансовая прозрачность. Автоматические напоминания об оплате, статистика доходов/расходов.

1.4 Анализ требований

Перед тем как приступить к проектированию системы, необходимо выявить все функциональные и нефункциональные требования [7], представленные к системе на основе опроса и конкурентов. Полученные требования отражают фактический функционал и технические особенности системы. Они включают в себя сильные стороны используемых аналогов, а также стремятся исключить повторения недостатков конкурентов.

Общие функциональные требования представлены на UML диаграмме рисунке Б.1.

Рассмотрим функциональные требования для пользователей с ролью преподавателя:

1. Создание групп с настройкой параметров (макс. количество учеников, стоимость занятий).
2. Составление гибкого расписания: повторяющиеся и разовые занятия. Перенос занятий.
3. Отметка посещаемости с возможностью добавления комментариев.
4. Возможность указать стоимость каждого занятия.
5. Введение статистики по доходам.
6. Управление контентом: добавление материалов к занятиям и домашнего задания.
7. Уведомление учеников о переносах или отмены занятий.

Для обучающихся:

1. Личный кабинет с расписанием и историей посещений занятий.
2. Вступление в группы преподавателей и просмотр открытых окон.
3. Возможность поиска преподавателей.

Далее рассмотрим нефункциональные требования:

- 1) удобство интерфейса. Современные паттерны применения дизайна, а также адаптивность под мобильные устройства;
- 2) безопасность. Надежное хранение персональных данных;
- 3) производительность. Возможность посещения платформы более 1000 человек одновременно;
- 4) поддержка внешних сервисов, таких как вход через Google или Yandex аккаунты.

1.5 Описание необходимого функционала

Для проектирования бизнес процессов выбрана методология описания схем IDEF0 [8]. Эта нотация является самой оптимальной для проведения описательных работ создания линейных бизнес процессов, требующие всеобъемлющего описания компонентов, элементов управления, действий, а также входных и выходных параметров.

Бизнес процесс авторизации в системе представлен на рисунке 1.10.

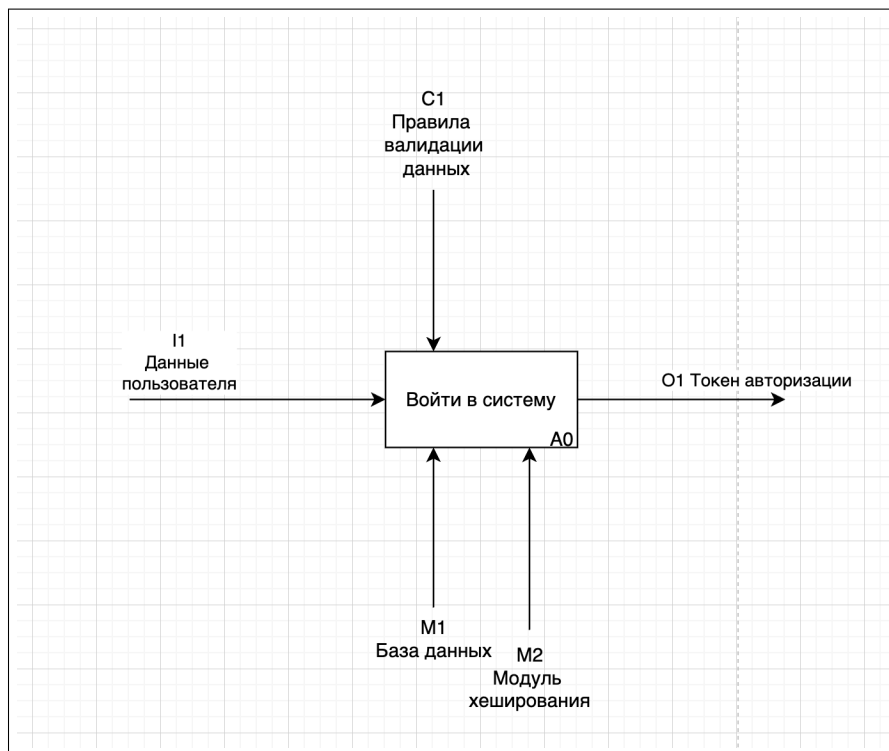


Рисунок 1.10 – Общий бизнес процесс входа в систему в нотации IDEF0

Этот процесс декомпозируется на регистрацию и авторизацию на уровень декомпозиции 1 и представлен на рисунке 1.11.

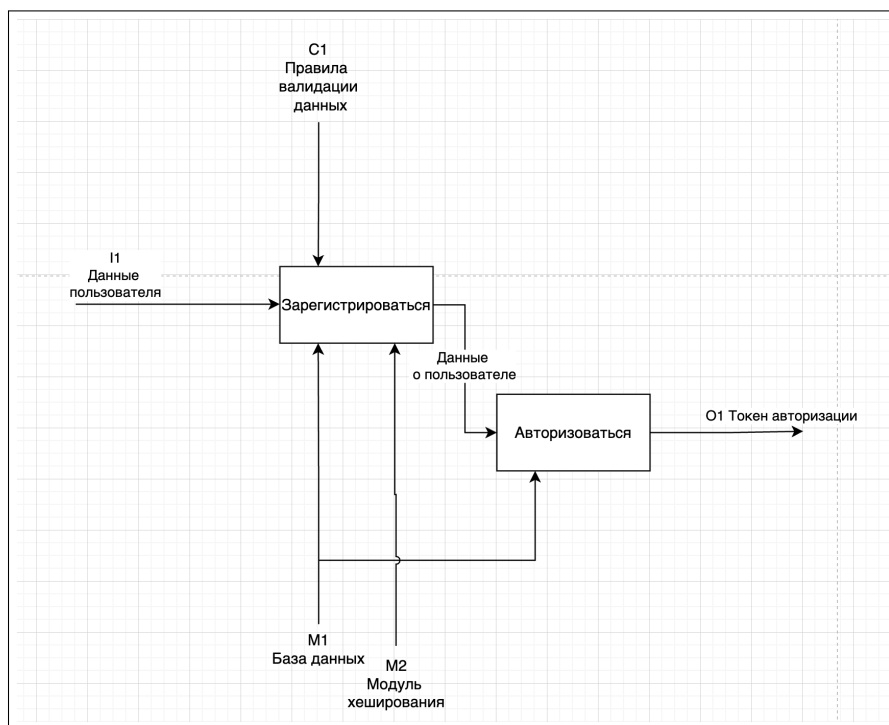


Рисунок 1.11 – Процесс входа в систему в нотации IDEF0 на 1 уровне декомпозиции

Далее, рассмотрим процесс регистрации, представленный на рисунке 1.12.

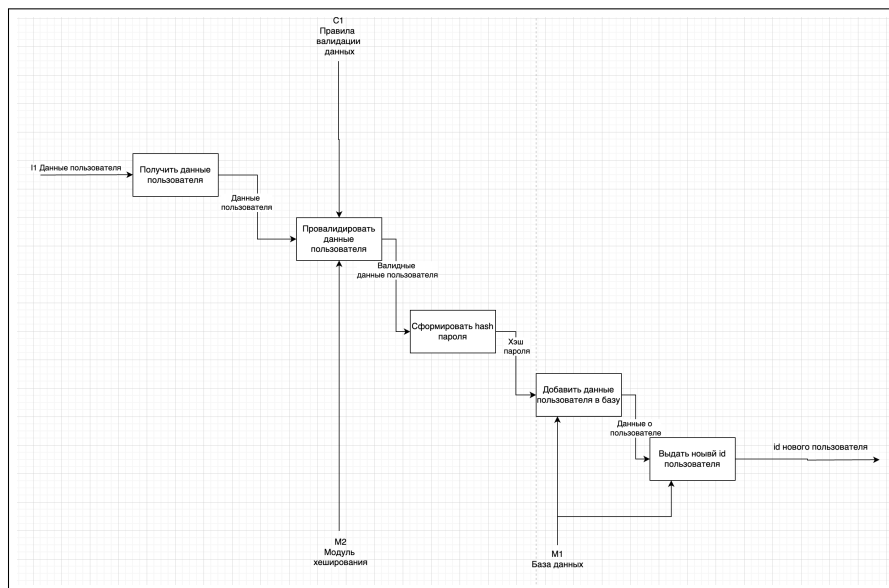


Рисунок 1.12 – Процесс регистрации пользователя в нотации IDEF0

Здесь, на вход поступают данные пользователя, а на выход id нового пользователя. Из механизмов здесь принимает участие модуль хеширования паролей, а также база данных в которой хранятся данные. Из правил, здесь присутствует Правила валидации полей.

В процессе регистарции существует процесс валидации данных, деком-позированный на рисунке 1.13.

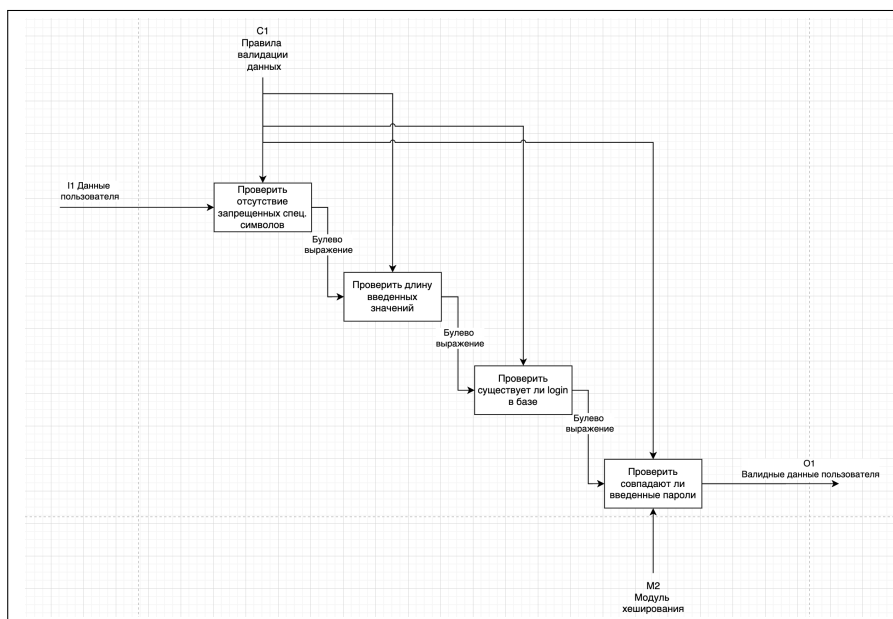


Рисунок 1.13 – Процесс валидации данных в нотации IDEF0

Здесь данные сначала валидируются по типу данных (строки числа или другое), далее проверяется существование введенного логина в базе данных, а также сверяются введенные пароли.

После того как пользователь зарегистрирован, ему необходимо авторизоваться под своим логином для доступа в систему.

Доступ в систему ограничен специальным токеном авторизации, который пользователь получает по факту авторизации и имеет срок истечения. По истечению срока действия токена, пользователю необходимо заново авторизоваться.

В процессе авторизации, представленном на рисунке 1.14, также используется процесс валидации введенных данных, после чего формируется уникальный идентификатор доступа в систему.

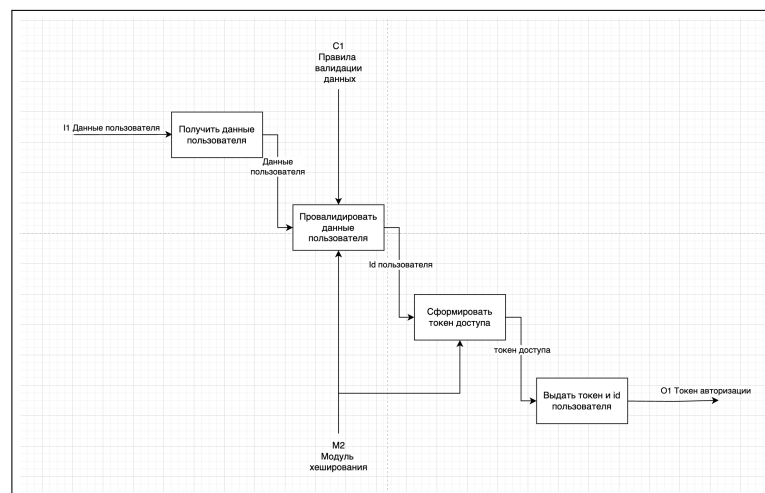


Рисунок 1.14 – Процесс авторизации в нотации IDEF0

Далее рассмотрим процесс вступления обучающихся в группы. Этот процесс представлен на рисунке 1.15 в виде диаграммы в нотации BPMN.

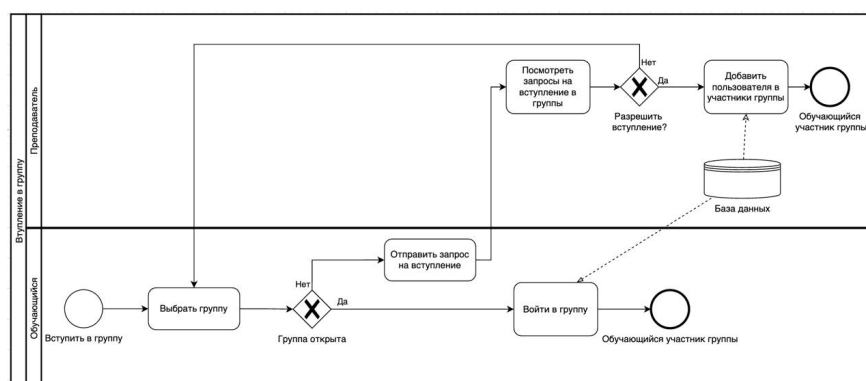


Рисунок 1.15 – Процесс вступления в группу в нотации BPMN

Здесь, пользователь, сначала выбирает желаемую группу из списка доступных групп. В случае если группа открыта, то у пользователя есть прямая возможность стать её участником. Этот процесс сопровождается добавлением новой записи в таблицу участников групп.

В случае если группа закрыта, создается запрос на вступление в выбранную группу. Создатель этой группы может просмотреть этот запрос, принять участника в группу или нет. Если да, пользователь также добавляется в таблицу участников группы, иначе запрос отклоняется, пользователю предложено выбрать новую группу.

При этом сам запрос администратор группы не может удалить. Его может удалить только отправитель запроса, в случае если он отзовет заявку на вступление в группу. Администратор также может передумать и спустя время принять заявку на вступление и впустить участника.

После вступления пользователя в группу, у ученика в расписании должны отображаться все действующие занятия, проводимые в группе. Расписание предусматривает отображение в виде текущего дня, выбранного дня, недельного представления, а также месячного представления.

В расписании, ученик может перейти в него и посмотреть актуальную информацию о предстоящем занятии.

Создатель занятия, он же создатель группы, может управлять текущими участниками занятия. Устанавливать присутствие или отсутствие, указать была ли проведена оплата, а также имеет возможность в таблице добавлять свои уникальные столбцы для каждого занятия, в которых он может указывать необходимую ему информацию. Например, это могут быть некие оценки - успеваемость по предметам, или учитывать комментарии по каждому участнику.

1.6 Выводы по аналитической части

В результате проведения аналитической работы были выполнены следующие задачи:

1. Проанализирована предметная область.
2. Получен портрет целевой аудитории.
3. Рассмотрены аналоги и составлены требования к системе.
4. Описаны функциональные бизнес процессы.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Анализ технических требований

Для реализации функциональных и нефункциональных требований, необходимо определиться с актуальным списком технологий. Поскольку итоговым продуктом является веб-приложение, то согласно статье, [9] современное веб приложение имеет следующие ключевые особенности в разработке:

1. **Отличие в отображении.** В отличие от традиционных приложений, веб-приложения функционируют в браузере и предоставляют пользователю взаимодействие с контентом на веб-страницах. Это связано с тем, что веб-приложения базируются на веб-технологиях, таких как HTML [10], CSS [11] и JavaScript [12], которые позволяют создавать интерактивные интерфейсы и обеспечивать гибкость в работе с различными устройствами.

2. **Разделение компонентов.** Веб-приложение как правило состоит из трех основных компонентов: клиентской стороны, серверной стороны и базы данных. Клиентская сторона отвечает за отображение информации пользователю и взаимодействие с ним, серверная сторона обрабатывает запросы клиента и принимает решения на основе логики приложения, а база данных хранит и обрабатывает данные, необходимые для работы приложения.

3. **Особенности работы.** Веб-приложения работают в режиме клиент-серверной архитектуры. Когда пользователь отправляет запрос, браузер обращается к серверу, который обрабатывает запрос и отправляет обратно отображаемые данные. Для этой цели используется протокол HTTP [13], который обеспечивает передачу данных между браузером и сервером. Таким образом, веб-приложения работают через сеть и доступны пользователю в любом месте, где есть доступ к интернету.

2.2 Проектирование базы данных

В качестве системы управления базой данных я выбрал PostgreSQL [14].

PostgreSQL - открытая российская система управления базами данных. Она базируется на языке SQL и поддерживает многие из возможностей стандарта SQL:2011 и ряд возможностей SQL:2016 в части работы с данными в формате JSON [15].

Сильными сторонами [16] PostgreSQL считаются:

- 1) высокопроизводительные и надёжные механизмы транзакций и репликации;
- 2) расширяемая система встроенных языков программирования: в стандартной поставке поддерживаются PL/pgSQL, PL/Perl, PL/Python и PL/Tcl, а также имеется поддержка загрузки модулей расширения на языке C;
- 3) наследование;
- 4) встроенная поддержка слабоструктурированных данных в формате JSON с возможностью их индексации;
- 5) расширяемость (возможность создавать новые типы данных, типы индексов, языки программирования, модули расширения, подключать любые внешние источники данных).

В PostgreSQL есть следующие ограничения:

- 1) максимальный размер таблицы: 32 Тбайт;
- 2) максимальный размер поля: 1 Гбайт;
- 3) максимум полей в записи: 250—1600, в зависимости от типов полей.

На рисунке 2.1 представлена общая даталогическая модель базы данных. Она включает в себя 11 таблиц и описывает связи между ними.

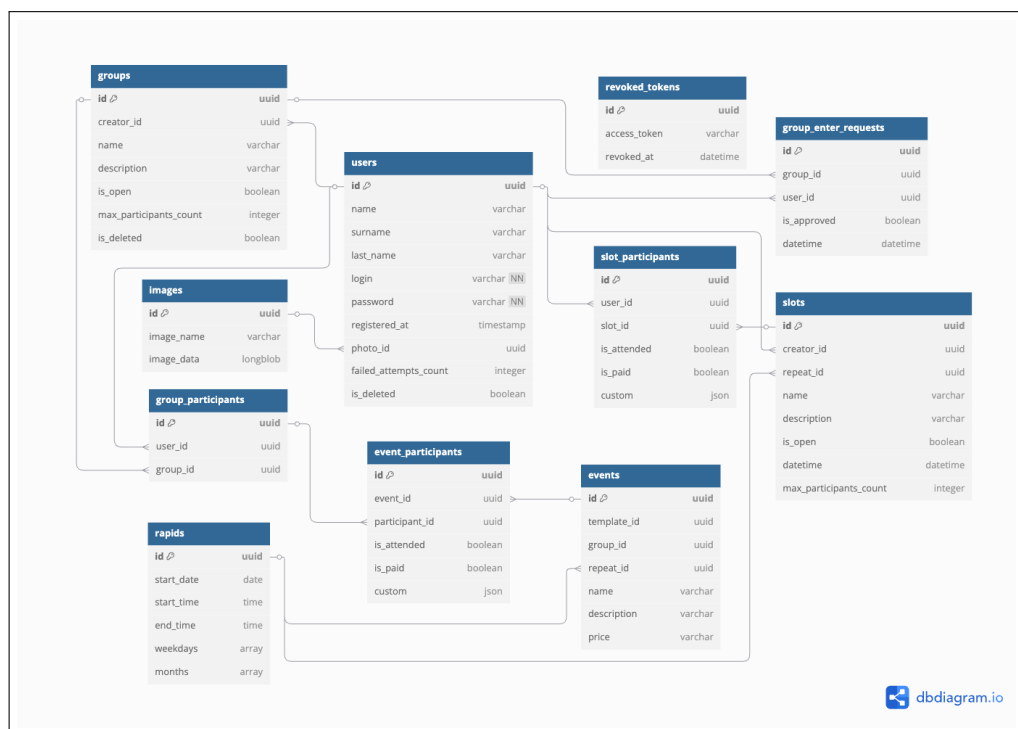


Рисунок 2.1 – Схема реляционной базы данных

Таблица о пользователях `users`. Содержит информацию о зарегистрированных пользователях в системе.

Основные поля таблицы:

- 1) `login` - уникальный логин авторизованного пользователя;
- 2) `password` - хешированный пароль;
- 3) `photo_id` - идентификатор фотографии;
- 4) `role_name` - имя роли.

Здесь поле `photo_id` является связью один к одному, поскольку у одного пользователя может быть одно изображение, и ни у кого другого пользователя быть не может. Информация об изображениях хранится в таблице `images`.

Таблица `images`. Содержит информацию об изображениях пользователей.

Основные поля таблицы:

- 1) `image_name` - название файла;
- 2) `image_data` - base64 формат картинки.

Таблица `groups`. Содержит информацию о группах - объединения пользователей по общему признаку.

Существуют группы открытые как для всех пользователей, так и с ограниченным доступом, за это отвечает поле `is_open`.

Все пользователи группы должны присутствовать на всех занятиях, которые создаются управляющим группы.

В открытые могут вступать абсолютно все пользователи, если количество участников группы меньше чем значение поля `max_participants_count`.

В закрытые можно вступить, отправив запрос на вступление, который рассмотрит создатель группы.

Основные поля таблицы:

- 1) `name` - название группы;
- 2) `max_participants_count` - максимальное количество участников;
- 3) `description` - описание;
- 4) `is_open` - флаг об открытости группы;
- 5) `creator_id` - id пользователя, который создал группу.

Пример SQL запроса на получение всех открытых групп, представлен на листинге 2.1.

Листинг 2.1 — Запрос на получение всех открытых групп

```
SELECT groups.id, groups.creator_id, groups.name, groups.description, groups.  
    is_open, groups.max_participants_count, groups.is_deleted  
FROM groups  
WHERE groups.is_deleted = false AND groups.is_open = true AND groups.  
    max_participants_count BETWEEN :max_participants_count_1 AND :  
    max_participants_count_2 AND groups.creator_id != :creator_id_1
```

Пример запроса на получение всех открытых групп, участником которых является авторизованный пользователь представлен на листинге 2.2

Листинг 2.2 — Запрос на получение всех открытых групп в роли ученика

```
SELECT groups.id, groups.creator_id, groups.name, groups.description, groups.  
    is_open, groups.max_participants_count, groups.is_deleted  
FROM groups  
JOIN group_participants ON groups.id = group_participants.group_id  
WHERE groups.is_deleted = false AND group_participants.user_id = :user_id_1  
    AND groups.is_open = false
```

Здесь необходимо делать JOIN [17] с таблицей group_participants в которой находится информация об участниках в группе

Таблица group_participants. Эта таблица содержит информацию об участниках группы. Это достигается путем хранения информации уникальных идентификаторов групп и пользователей, для определения кто в какой группе состоит или не состоит.

Пользователи могут состоять в сразу нескольких группах.

Поля таблицы group_participants:

- 1) group_id - уникальный идентификатор группы;
- 2) user_id - уникальный идентификатор пользователя.

Таблица group_enter_requests. Содержит информацию о запросах пользователей на вступление в закрытые группы.

После добавления пользователя в группу, информация о нем заносится в таблицу group_participants.

Основные поля таблицы group_enter_requests:

- 1) user_id - id пользователя, который отправил запрос;
- 2) group_id - id группы, в которую пользователь хочет вступить;
- 3) datetime - дата и время создания запроса;
- 4) is_approved - флаг о принятом или отклоненном запросе.

Состояния поля `is_approved`:

- 1) `null` - запрос еще не рассмотрен;
- 2) `False` - запрос не удовлетворен;
- 3) `True` - запрос удовлетворен и пользователь заносится в таблицу участников.

Пример SQL запроса на получение всех заявок на вступление в группу представлен на листинга 2.3

Листинг 2.3 — Запрос на получение всех заявок на вступление в группу

```
SELECT group_enter_requests.id, group_enter_requests.user_id,  
       group_enter_requests.group_id, group_enter_requests.datetime,  
       group_enter_requests.is_approved  
FROM group_enter_requests  
WHERE group_enter_requests.group_id = :group_id_1
```

Таблица `events`. Эта таблица содержит информацию о всех занятиях внутри одной группы. Занятия могут повторяться, каждому указана стоимость, а также существует внешняя связь по участникам каждого занятия. Эта связь указывает сразу на всех участников одной группы. При создании занятия, создатель группы указывает описание и название занятия, стоимость, а также частоту повторения.

Основные поля таблицы:

- 1) `group_id` - группа, в которой это событие происходит;
- 2) `repeat_id` - ссылка на таблицу `rapids` с указанием частоты;
- 3) `name` - название занятия;
- 4) `description` - описание занятия;
- 5) `price` - стоимость занятия(опциональное поле);
- 6) `event_participants` - связь с таблицей участников занятия - является массивом всех участников одного занятия. Связь - один ко многим.

Здесь для упрощения работы с клиентской частью, в базу данных идет следующий SQL запрос который получает все поля из таблицы `events` и таблицы `rapids`, соединяя их в одну запись.

Пример этого запроса для авторизованного пользователя с ролью учителя для получения всех занятий по своим группам в промежутке между двумя датами представлен на листинге 2.4.

Листинг 2.4 — Запрос на получение всех занятий по всем своим созданным группам

```
SELECT events.id, events.name, events.description, events.group_id, events.  
    repeat_id, events.price  
FROM events  
JOIN rapids ON events.repeat_id = rapids.id  
JOIN groups ON events.group_id = groups.id  
WHERE groups.creator_id = :creator_id_1 AND rapids.start_date BETWEEN :  
    start_date_1 AND :start_date_2
```

Здесь идет JOIN с таблицей rapids для получения данных из этой таблицы, а также JOIN с таблицей groups для добавления фильтра по пол. creator_id

Также, для учащегося, происходит немного другой запрос. Для него в первую очередь сначала нужно определить в каких группах он принимает участие, и только по ним доставать занятия. Пример такого SQL запроса в котором учащийся получает все занятия из групп в которых он состоит представлен на листинга 2.5

Листинг 2.5 — Запрос на получение всех занятий по всем группам

```
SELECT events.id, events.name, events.description, events.group_id, events.  
    repeat_id, events.price  
FROM events  
JOIN rapids ON events.repeat_id = rapids.id  
JOIN group_participants ON events.group_id = group_participants.group_id  
WHERE group_participants.user_id = :user_id_1 AND rapids.start_date BETWEEN :  
    start_date_1 AND :start_date_2
```

Здесь также используется JOIN с таблицей rapids по полю repeat_id, а также идет JOIN с таблицей group_participants для получения информации о том, что пользователь является участником.

Таблица rapids. Является вспомогательной таблицей для таблиц events и slots. В ней хранится информация о том когда каждое занятие начинается, заканчивается, по каким дням и месяцам оно повторяется. Основные поля

- 1) weekdays - массив из цифр, дней недели по которым занятие или слот повторяется;
- 2) months - массив из цифр, номеров месяцев по которым занятие повторяется;
- 3) start_date - строка из даты в которое начинается занятие;
- 4) start_time - время в которое начинается занятие;

- 5) `end_time` - время в которое занятие закончится;
- 6) `events` - связь, массив из занятий в которых эта запись используется;
- 7) `slots` = связь, массив из слотов в которых эта запись используется.

Таблица `event_participants`. Информация об участниках одного конкретного события.

О каждом участнике ведется следующая информация: его присутствие, оплачена занятия, а также другие индивидуальные параметры. Поля в таблице:

- 1) `event_id` - id события, участником которого является данный участник. Здесь связь 1 ко многим, потому что у занятия может быть много участников;
- 2) `participant_id` - id участника группы, которым также является участник события. Здесь связь 1 ко многим, потому что участник группы может быть участником нескольких занятий;
- 3) `is_attended` - флаг о том что участник группового занятия пришел на него;
- 4) `is_paid` - флаг о том, что участник группового занятия оплатил его;
- 5) `custom` - поля типа JSON, которое содержит уникальную информацию по каждому участнику группового занятия. Например, его полученную оценку или другие.

Таблица `slots`. Содержит информацию об открытых занятиях вне групп. Их назначение схоже с назначением событий, но отсутствует связь с группами. Каждый слот - является одновременно и группой, и занятием.

Основные поля:

- 1) `creator_id` - ID пользователя, который создал этот слот;
- 2) `repeat_id` - связь с таблицей `repeats`;
- 3) `name` - название;
- 4) `description` - описание;
- 5) `max_participants_count` - число, максимальное количество пользователей которые могут участвовать в слоте;
- 6) `price` - цена;
- 7) `participants` - связь с таблицей `slot_participants` - массив из участников этого слота.

Пример SQL запроса для получения всех своих созданных слотов или слотов любого пользователя представлен на листинге 2.6. Он актуален для всех авторизованных пользователей с ролью учителя.

Листинг 2.6 — Запрос на получение всех слотов одного пользователя

```
SELECT slots.id, slots.creator_id, slots.repeat_id, slots.name, slots.  
description, slots.max_participants_count, slots.price  
FROM slots JOIN rapids ON slots.repeat_id = rapids.id  
WHERE slots.creator_id = :creator_id_1 AND rapids.start_date BETWEEN :  
start_date_1 AND :start_date_2
```

Таблица `slot_participants`. Содержит информацию об участниках открытых событий. По ним также есть возможность указывать оплату, посещение и вносить свои уникальные кастомные характеристики по каждому участнику слота.

Поля таблицы:

- 1) `id` - Уникальный идентификатор участника слота;
- 2) `user_id` - ID пользователя - участника одного слота;
- 3) `slot_id` - ID слота, участником которого является пользователь;
- 4) `is_attended` - флаг о присутствии участника на этом слоте;
- 5) `is_paid` - флаг об оплате участником этого слота;
- 6) `custom` - поле типа JSON, которое содержит уникальную информацию по каждому участнику группового занятия. Например, его полученную оценку на занятии или комментарий, оставленный преподавателем. Информация хранится в паре ключ-значение.

2.3 Проектирование и разработка серверной части приложения

Прежде чем провести обзор архитектуры серверной части приложения, необходимо рассмотреть программные инструменты, причины их выбора, а также провести анализ основных технологий, которые используются для создания серверных частей веб-приложений.

Современные веб-приложения используют разнообразные языки программирования и архитектурные подходы для реализации серверных компонентов [18].

Сравнительная характеристика самых популярных языков программирования, используемых в серверной части представлена в таблице 2.1.

Таблица 2.1 - Сравнительная характеристика технологий на серверной части.

Язык программирования	Преимущества	Недостатки
PHP	Огромное наследие, множество готовых решений (WordPress, Laravel) Низкий порог входа Хорошая документация	Проблемы с производительностью в высоконагруженных системах Нестрогая типизация
Go	Высокая производительность Простая конкурентная модель (горутины) Статическая компиляция	Меньше библиотек по сравнению с Python/Node.js Строгий синтаксис (меньше гибкости) Слабее в быстрой разработке прототипов
NodeJS	Асинхронная модель из коробки Один язык для frontend и backend Большая экосистема (npm)	Callback hell (хотя есть async/await) Слабая типизация (TypeScript решает частично) Проблемы с CPU-интенсивными задачами
Python	Читаемый и выразительный синтаксис Богатейшая экосистема (Django, Flask, FastAPI)	Медленнее компилируемых языков Динамическая типизация (но есть type hints)

В качестве основного языка программирования выбран Python. Для создания функционального веб сервиса выбран фреймворк FastAPI [19]. Выбор обусловлен следующими факторами:

- 1) быстрая разработка: Python позволяет создавать прототипы приложений максимально быстро;
- 2) современный подход: FastAPI предоставляет документацию OpenAPI, встроенную валидацию через Pydantic, поддержку async/await;
- 3) гибкость: возможность интеграции с ML-моделями, аналитическими инструментами и внешними сервисами;
- 4) сообщество: огромное количество готовых решений и библиотек;
- 5) производительность: FastAPI сравним по скорости с Node.js и Go в типичных web-задачах.

Python и FastAPI — это оптимальный баланс между скоростью разработки, производительностью и поддерживаемостью кода для учебного проекта.

Определившись с списком технологий, рассмотрим архитектуру серверной части приложения. На рисунке 2.2 представлена MVC [20] схема движения данных внутри сервера.

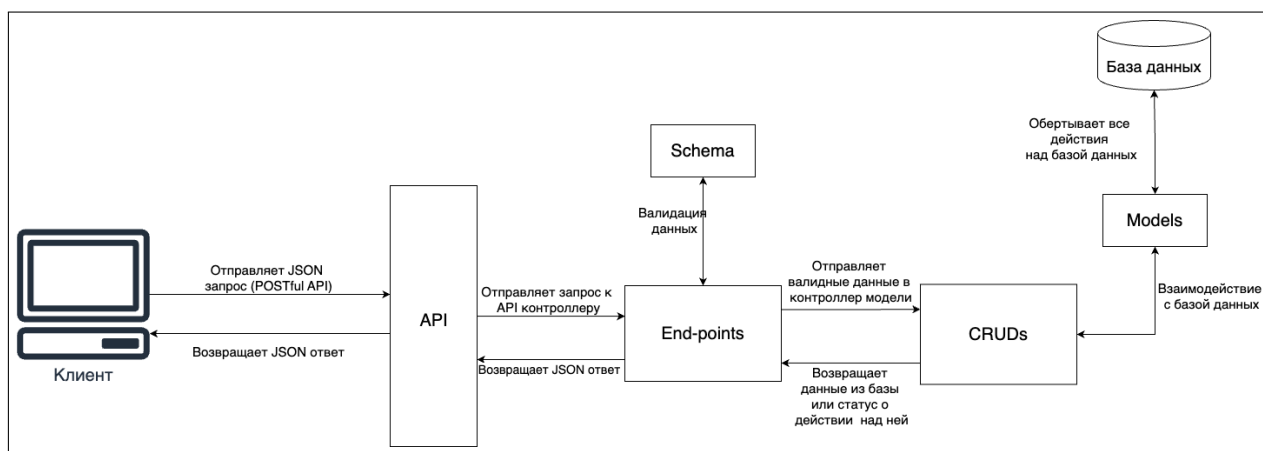


Рисунок 2.2 – Схема взаимодействия данных от клиента до базы данных

С клиента поступает JSON запрос на получение, изменение, удаление или создание определенных данных.

По указанному запросу выполняется одна точка входа в серверное приложение. Прежде чем работать с данными - их необходимо отвалидировать, проверить, что данные удовлетворяют требованиям. За это отвечают схемы валидации - Schemas на диаграмме. В случае если данные не валидны - API возвращает 404 [21] ошибку пользователю, иначе продолжит выполнение операции.

В зависимости от точки входа, выполняется бизнес логика взаимодействия с базой данных. Она происходит в части CRUD [22] на диаграмме. CRUD - акроним от Create Read Update Delete, что переводится как создание, чтение, обновление и удаление соответственно. В так называемых "крудах" происходит непосредственное взаимодействие с моделями. Модель - ORM [23] представление таблицы в базе данных, обеспечивающая упрощенный доступ к взаимодействию с данными. В отличие от схемы, модель позволяет не только отвалидировать данные, но предоставляет функционал к генерации SQL запросов к базе данных.

Полученные данные из базы, дополнительно форматируются на уровне "крудов", для необходимого формата чтения данных клиентской частью и передаются обратно в контроллер API - "эндпоинт". В контроллере происходит повторная валидация с помощью схем, чтобы удостовериться, что получен-

ные данные из базы данных находятся в ожидаемом формате для клиентской части. После валидации серверная часть возвращает ответ к клиенту в формате JSON.

Далее, рассмотрим используемые модули для обеспечения внутреннего взаимодействия классов серверной части приложения, касающиеся движения данных от эндпоинта до базы данных.

Идеальным ORM поставщиком является `sqlAlchemy` [24] - позволяет использовать модельный архитектурный подход, в котором каждая таблица в базе данных представлена моделью с которой работает система. Это позволяет не использовать чистый SQL, а оперировать встроенным функционалом запросов, методы `query` для составления сложных SQL запросов.

На слое валидации я использую схемы от модуля `Pydentic` [25]. Этот модуль позволяет не описывать входные и выходные параметры в `yaml` файлах, вся валидация происходит в специальных классах - схемах, где указываются типы полей и `pydentic` схема сама валидирует введенные и возвращаемый параметры данных. Зачастую они схожи с `sqlAlchemy` моделями и поэтому их описание занимает гораздо меньше времени.

В качестве системы контроля версий был выбран `github` [26].

Рассмотрим вспомогательные компоненты для взаимодействия по HTTP. На уровне API, первой точкой входа это "end-поинты". Они являются публичными методами, которые использует клиентская часть проекта.

Для оптимизации временных затрат я принял решение использовать `POSTful API` [27]. Это позволяет не разделять логику процессов на различные типы запросов. Любое получение, изменение и создание сущностей в базе данных происходит через `POST` запросы. Они имеют закрытое тело запроса в формате JSON, обеспечивая поддержку любого типа данных (будь это строки, числа, массивы или объекты).

В моем проекте, в папке `endpoints` лежат все файлы относящиеся к сущностям. Например, `enter_requests` отвечает за принятие и отклонение запросов на вступление в закрытые группы пользователей.

Однако, с увеличением количества отдельных модулей возникает необходимость в оптимизации процесса добавления этих модулей в `__init__` файл. Этот файл позволяет использовать все внутренние файлы папки как один модуль, ссылаясь на папку `endpoints` целиком.

Для того чтобы избежать ручной ввод путей в этот файл, а также чтобы избежать появление ошибок внутри бизнес логики, что привело бы к остановке приложения в принципе, я разработал алгоритм, интегрирующий маршруты к точкам входов каждого "эндпоинта" в один общий маршрутизатор, используемый FastAPI приложением. Этот алгоритм представлен на листинге 2.7.

Листинг 2.7 — Алгоритм сбора маршрутов

```
1  """
2  Entry point for all endpoints
3  """
4  import os
5  import importlib
6  from fastapi import APIRouter
7  from env_constants import BASE_URL
8  app_router = APIRouter()
9  def include_routers(app_router: APIRouter):
10     endpoints_path = os.path.join(os.path.dirname(__file__), 'endpoints')
11     for filename in os.listdir(endpoints_path):
12         if filename.endswith('.py') and filename != '__init__.py':
13             module_name = filename[:-3]
14             try:
15                 module = importlib.import_module(
16                     f'api.endpoints.{module_name}')
17                 if hasattr(module, 'router'):
18                     app_router.include_router(
19                         module.router, prefix=f'/{BASE_URL}', tags=[
20                             module_name])
21                     print(f'{module} router is added')
22             except Exception as e:
23                 print(f'Failed to include router from {filename}: {e}')
24 include_routers(app_router)
```

Разберемся более детально в коде листинга 2.7.

Перед тем как вызвать функцию агрегации маршрутов API, создается общий FASTAPI роутер в который последовательно добавляются все роутеры из модулей в папке endpoints.

Первостепенно происходит получение абсолютного пути к папке в которой хранятся "эндпоинты" (папка endpoints) на строке 10. Это достигается благодаря использованию встроенной библиотеки os (строка 4) и объединения абсолютного пути с названием папки.

Далее происходит запуск алгоритма итерации по списку всех файлов внутри директории "эндпоинтов" (строка 11).

В случае если имя файла содержит в себе подстроку `__init__` этот файл пропускается и процесс итерации продолжается далее (строка 12).

Далее с помощью библиотеки `importlib` я динамически загружаю содержимое файла в оперативную память (строка 15). В случае если в модуле содержится переменная `router`, я включаю его в свой общий роутер.

Таким образом, область включения маршрутов включает в себя все существующие маршруты без необходимости вручную импортировать их в общий инициализационный файл.

Далее, если один из роутеров содержит в себе ошибку, сработает обработчик `Exception` (строка 21) и выведет в консоль трэйс ошибки и причину по которой маршрут не смог собраться.

Далее в приложении уже используется обще собранный роутер со всеми маршрутами к эндпоинтам.

Каждый роутер использует бизнес логику из отдельных файлов в папке `cruds`. Каждый из них наследуется от базового круда.

Рассмотрим файл `crud/base.py`, листинг которого представлен в приложении В листинг 1. В нем описан класс `BaseCRUD`, в котором существуют основные методы взаимодействия с базой данных.

Как было упомянуто ранее, `CRUD` [28] - является акронимом от производных слов: `Create`, `Read`, `Update`, `Delete` - создание, чтение, обновление и удаление соответственно. Правда в моем случае, стоит добавить еще букву `L` - `List` - подтип метода `read` - чтения сразу нескольких элементов из базы данных.

Данный класс является общим и напрямую никогда не вызывается. От него наследуются [29] другие классы `CRUD`-ы, которые оперируют с одной или с несколькими моделями сразу. Мой базовый круд поддерживает одновременную работу как с одной моделью, так и с несколькими.

Рассмотрим по отдельности все основные публичные этого класса. Круды наследники могут переопределять и использовать все методы, а уже классы контроллеров могут использовать только публичные методы.

Инициализация класса. По умолчанию, при инициализации базового круда, ему необходимо передать:

- 1) `db` - экземпляр сессии базы данных;
- 2) `model` - основная модель с которой базовый круд будет работать и обращаться запросы к ней;
- 3) `joined_models` - список моделей с полями для фильтрации.

Публичный метод получения одной записи. Метод `get_item` предназначен для получения одной записи из таблицы, указанной при инициализации класса или в параметрах вызова.

На вход он получает `body` запроса - как правило там только `id` записи, `field` - строка, поле по которому будет проходить поиск, по умолчанию `id`.

Сперва метод создает экземпляр `query` - запроса (`SELECT * FROM model`) на который будут накладываться другие фильтры.

Для начала следует проверить, что переданное поле `field` описано в модели, а также о том, что в `body` запросе это значение существует. Если нет, то возвращается 404 ошибка `NotFoundError`.

Далее, происходит попытка получения объекта модели с помощью применения фильтра `wheres` к `query` по полю которое получено из тела запроса и значению из тела запроса запроса.

Полученный объект возвращается.

Метод `get` использует метод `get_item`, с разницей лишь в том, что полученный объект он обарачивает через наследуемый метод `_make_output` - для кастомного преобразования полученного элемента.

Метод получения списка данных. Метод `list` используется для получения одного или нескольких значений из базы данных. На вход поступает `POST` запрос с телом в формате `JSON`, в котором данные сформированы в специальном формате фильтров и сортировок.

Метод вызывает публичный метод `get_items` и полученный результат также обарачивает в метод `_make_output` - для кастомного преобразования каждого полученного элемента, а затем в необходимый формат для клиентской части.

Метод `get_items` же, сначала получает массив фильтров из тела запроса - как правило это массив из объектов, в котором `column` - это поле по которому будет проходить фильтрация, `value` - это значение с котормы необходимо сравнить, а `condition` - это тип сравнения значения `value` с значениями в базе данных.

В коде базового круда реализовано несколько типов сравнений:

- 1) `==` полное сравнение, устанавливается по умолчанию;
- 2) `!=` отрицательное сравнение, не равно переданному значению;
- 3) `in` значение встречается частично в базе - для массивов и объектов;
- 4) `%` строка встречается в другой строке;
- 5) `between` значение находится между двух значений в массиве `value`.

Этот метод к `query` применяет поочередно все фильтры, делает запрос к базе данных и возвращает все совпадения.

Метод обновления записи. Метод `update` сначала находит элемент из тела запроса в базе данных. Если такого нет, то возвращается 404 ошибка, иначе далее по каждому полю которое передано в теле запроса, достается значение, проверяется что оно не пустое и что поле не является неизменяемым. Если эти условия выполняются, то создается правильный валидный экземпляр который подлежит обновлению в базе данных.

Рассмотрев класс, позволяющий работать с базой данных и воплощать логику отдельных "эндпоинтов", рассмотрим отдельный файл, позволяющий локально авторизовывать и регистрировать пользователей.

Рассмотрим файл `crud/auth.py`, листинг которого представлен в приложении В листинг 2.

В нем описан класс `Authorisation` регистрации, авторизации и прочих манипуляций с базой данных для предоставления функционала авторизованного пользователя.

Для начала рассмотрим список ключевых используемых библиотек и модулей:

- 1) `sqlalchemy` - ORM работа с базой данных;
- 2) `models` - классы моделей для работы с базой данных;
- 3) `schemas` - Схемы валидации данных на вход и выход;
- 4) `fastapi` - `Depends` - класс установки зависимости для эндпоинтов;
- 5) `BaseCRUD` - Базовый круд;
- 6) `exc` - кастомные исключения, отправляемые клиенту;
- 7) `jose` - библиотека для работы с JWT токенами;
- 8) `env_constants` - конфигурационные константы;
- 9) `bcrypt` - библиотека для шифрования паролей.

Рассмотрим существующие методы и функции.

Зависимость авторизованного доступа. Функция `authorised_user` обеспечивает только авторизованный доступ к эндпоинтам внутри системы. В случае если пользователь будет пытаться получить любую информации из защищенных эндпоинтов без токена авторизации, ему будет возвращен 401 код ошибки `Unautherized`.

На вход функция получает `token` - зашифрованный токен авторизации, при расшифровке которого получается информация о дате его выдаче и о логине пользователя которому он был выдан. На выход функция либо возвращает экземпляр авторизованного пользователя, либо возвращает исключение - 401 ошибка код авторизации.

Токен авторизации пользователь передает в хедерах части запроса.

Первым делом, функция обращается к классу `Authorisation` и вызывает метод `is_token_revoked`.

Этот метод обращается к таблице `revoked_tokens` и получает все токены авторизации которые уже были завершены и более не являются действительными, и пытается найти там токен предоставленный пользователем.

Если токен пользователя найден среди отозванных, то предоставленный токен уже недействителен и пользователю возвращается соответствующее сообщение с 401 ошибкой.

Иначе, если токен не был отозван, то следующим шагом, токен расшифровывается с помощью библиотеки `jwt` - метод `decode`. Он расшифровывается с учетом секретного кода, который устанавливается при запуске приложения в `env` файле окружения. Для упрощения работы, я использовал вместо него, пароль от базы данных к приложению.

Из полученной сигнатуры пользователя `payload` можно получить `login` пользователя, JWT [30] токен которого у нас получен.

Далее, если JWT токен невалиден, а также `login` не получен, то этот токен заносится в таблицу `revoked_tokens` и на клиент возвращается 401 ошибка авторизации.

Иначе, по полученному `login`-у пользователя, идет обращение к базе данных для поиска пользователя с таким логином. Если такого пользователя не существует, клиенту возвращается соответствующая ошибка с 401 кодом, иначе возвращается информация об авторизованном пользователе.

Соответственно, если метод выполнен успешно и без ошибок, то токен авторизации валиден, пользователю разрешено использование других методов API.

Регистрация. Метод `register` класса `Authorisation`, который отнаследован от класса `BaseCRUD`, получает как входной параметр экземпляр класса `UserCreate` - объект с полями `login`, `password`, `password_confirm`.

Получив значения, для начала метод проверяет существует ли уже в базе пользователь с полученным логином. Если нет, то необходимо проверить, что полученные два пароля совпадают друг с другом.

Если пароли совпадают и такого пользователя нет в базе данных, создается новый хеш пароль [31] полученного пароля, для того чтобы сохранить только хеш в базе данных.

Метод хеширования паролей. Функция `_hash_password` принимает на вход строку - пароль в явном виде и возвращает хешированный пароль с помощью библиотеки `bcrypt`.

Он генерирует пароль с генерацией специальной соли - случайной величины которая гарантирует безопасность хранения паролей пользователей.

Авторизация. Метод авторизации `login` принимает на вход экземпляр класса `UserLogin` - объект из двух полей `login` и `password`.

Для начала, необходимо проверить, что пользователь существует и что переданный пароль совпадает с тем, что хранится в базе данных. - метод `_verify_token`.

Если этот метод возвращает экземпляр авторизованного пользователя, то необходимо вернуть этот объект пользователю, а также сформировать новый токен авторизации с которым далее пользователь сможет получать данные по другим эндпоинтам.

Этот алгоритм прост, для начала необходимо определить время когда этот токен станет невалидным. С помощью этого значения, передается в метод `_create_access_token` - полученный токен далее возвращается пользователю.

Метод проверки на существование пользователя и совпадение паролей. Метод `_verify_token` получает на вход `login` и `password` пользователя, статус авторизации которого пока не определена. Для начала получаем информацию о пользователе из базы данных. Если она есть, значит пользователь существует и можно сравнивать пароли. Иначе возвращается 401 ошибка.

Пароли сравниваются следующим образом: По полученному прямому паролю пользователя создается хеш с солью и сравнивается с тем, что хранится в базе данных. Если эти хеши совпадают - пользователю гарантирован вход, иначе 401 ошибка.

2.4 Разработка интерфейса

Рассмотрим процесс создания уникальной айдентики приложения.

В рамках разработки веб-приложения администрирования групповых занятий особое внимание было уделено оформлению пользовательского интерфейса, цветовая палитра [32] формирует у пользователя визуальной идентичности проекта. Сформированная палитра представлена на рисунке 2.3.

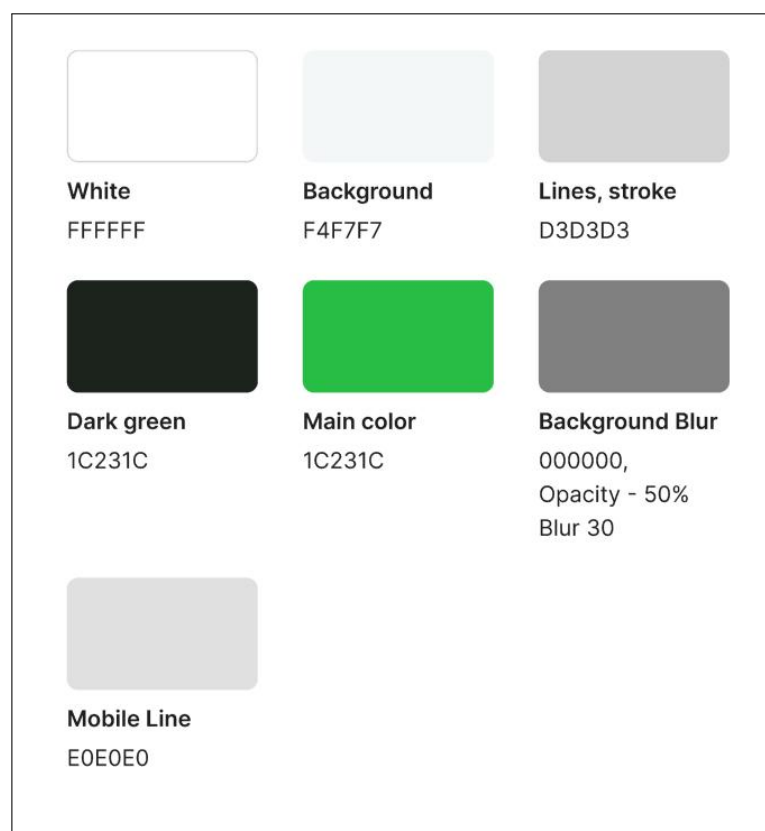


Рисунок 2.3 – Цветовая палитра

Основной цвет: Темно-зеленый (#1C231C). Используется для заголовков, ключевых кнопок и важных элементов интерфейса. Этот глубокий оттенок зелёного ассоциируется с стабильностью, профессионализмом и экологичностью, что делает его идеальным для образовательных платформ. Он обеспечивает хороший контраст на светлом фоне и помогает пользователям быстро находить ключевые зоны взаимодействия.

Фоновый цвет: Светло-серый (#F4F7F7). Применяется в качестве основного фона интерфейса. Его нейтральность и мягкость снижают визуальную нагрузку, создавая комфортную среду для работы с контентом. Этот цвет также улучшает читаемость текста и других элементов, размещённых на нём.

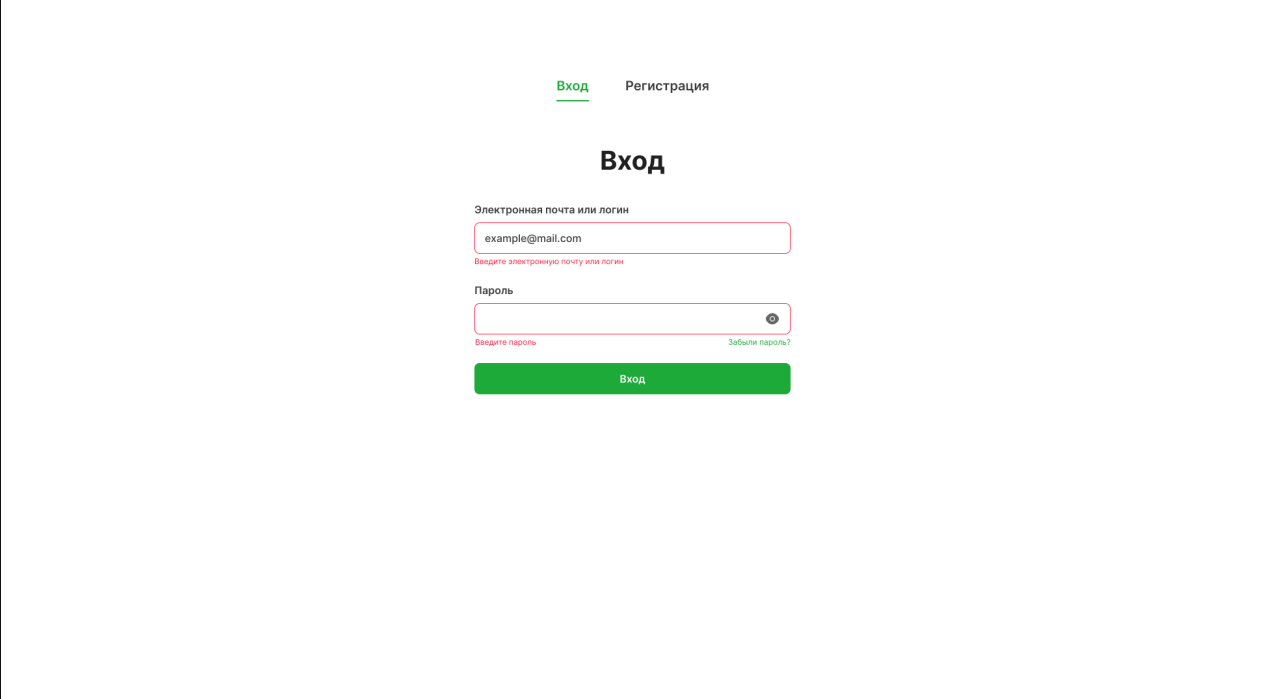
Границы и разделители: Светло-серый (#D3D3D3). Используется для линий, рамок и визуального разделения блоков (например, между строками в расписании или секциями меню). Его лёгкость помогает структурировать информацию, не перегружая интерфейс.

Мобильные разделители: Светло-серый (#E0E0E0). Оптимизирован для мобильных устройств — применяется для тонких линий в компактных представлениях (например, в списках учеников или настройках). Чуть светлее, чем #D3D3D3, чтобы лучше сочетаться с небольшими экранами.

Фоновое размытие: Чёрный с прозрачностью (#000000, 50% opacity + blur 30). Необходимо для модальных окон, всплывающих подсказок и затемнения фона. Размытие помогает сфокусировать внимание на активном элементе (например, форме добавления урока), а прозрачность сохраняет связь с контекстом.

Далее, рассмотрим процесс создания макетов страниц.

На рисунке 2.4 представлен макет страницы для входа в систему.



Вход Регистрация

Вход

Электронная почта или логин

Введите электронную почту или логин

Пароль

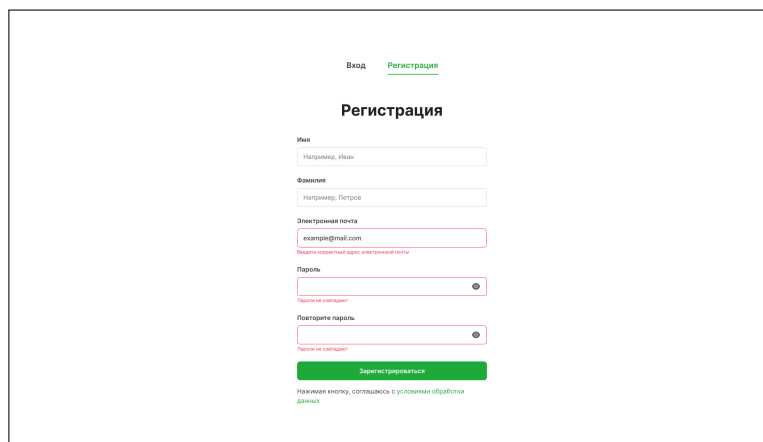
Введите пароль Забыли пароль?

Вход

Рисунок 2.4 – Прототип страницы Вход

Это форма с двумя полями, у каждого поля для ввода есть подсказки, а также указаны ошибки если введенные значения некорректны.

На рисунке 2.5 представлен макет страницы для регистарции в системе.



Вход [Регистрация](#)

Регистрация

Имя

Фамилия

Электронная почта
Введите корректный адрес электронной почты

Пароль
Пароль не совпадает

Повторите пароль
Пароль не совпадает

[Зарегистрироваться](#)

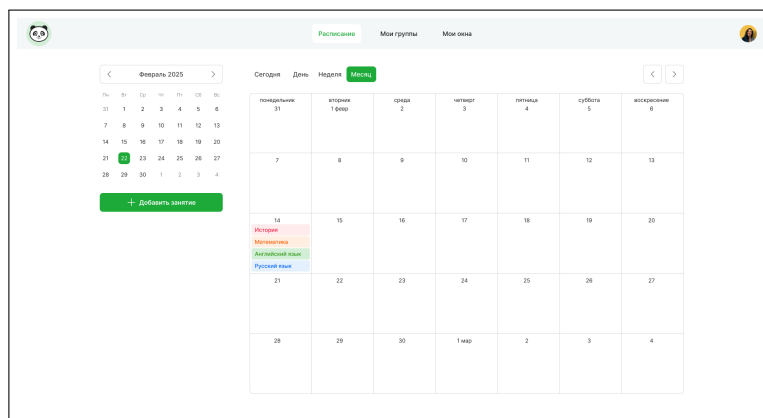
Нажимая кнопку, соглашаюсь с условиями обработки данных

Рисунок 2.5 – Прототип страницы Регистрация

В форме представлены основные поля необходимые для регистрации нового пользователя. При вводе несовпадающих парой, высвечивается соответствующее сообщение о валидации. Также, под кнопкой о регистрации можно найти ссылку на условия обработки персональных данных.

На рисунке Б.2 представлен макет страницы расписания пользователя - одного из 3 основных разделов сервиса. На вкладке расписание находится интерактивный компонент с календарем, позволяющий переключаться между месяцами, а также кнопкой создания занятия на выбранную дату.

Справа от него находится более подробный компонент календаря с переключением между днем (рисунок Б.2), неделями (рисунок Б.3) и месяцем (рисунок 2.6).



Расписание | Мои группы | Мои занятия

Февраль 2025

Сегодня	День	Неделя	Месяц			
понедельник 31	вторник 1 февр	среда 2	четверг 3	пятница 4	суббота 5	воскресенье 6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1 март	2	3	4

Добавить занятие

Рисунок 2.6 – Прототип страницы "Расписание". Календарь, месяц

В каждом из них в удобном формате представлены предстоящие занятия среди всех групп. На рисунке 2.7 представлен макет страницы групп с точки зрения Преподавателя.

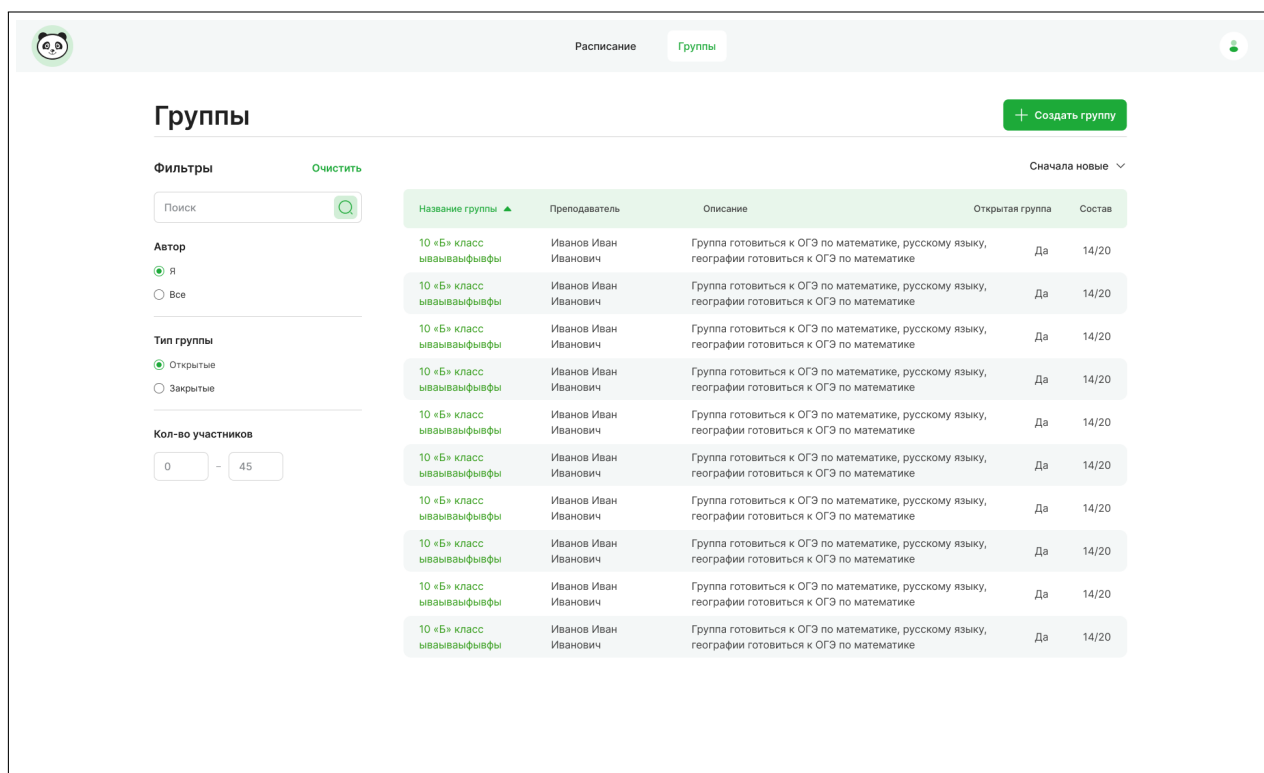


Рисунок 2.7 – Прототип страницы таблицы групп от лица преподавателя

Здесь, в левой части страницы располагается форма для фильтрации групп по характеристикам:

1. По автору группы: им является текущий пользователь или другие пользователи.
2. По области видимости: будут отображаться группы с свободным входом или с одобрением администратора.
3. По количеству участников в группе.

Справа от формы располагается сама таблица с полями: Название, Преподаватель, Описание группы, является ли она открытой или закрытой, а также её состав - общее количество участников к максимальному.

Над таблицей можно наблюдать кнопку создания новой группы. Она видна только пользователю с ролью преподавателя.

При клике на группу в таблице, мы переходим в карточку группы, представленную на рисунке 2.8.

Рисунок 2.8 – Прототип страницы редактирования группы от лица преподавателя

Рисунок 2.8 – Прототип страницы редактирования группы от лица преподавателя

Если пользователь является её автором, он может отредактировать информацию о группе, а также ему предоставляется функционал управления участниками, занятиями, а также запросами на вступление, если группа является закрытой.

На рисунке 2.9 представлен прототип страницы просмотра участников группы.

ФИО участника	Почта
☐ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☐ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☒ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☐ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☒ Иванов Иван Иванович	ivanoff11247348@gmail.com
☒ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com

Исключить

Рисунок 2.9 – Прототип страницы управления участниками группы от лица преподавателя

Здесь, пользователь может прямо в таблице выбрать участников, чтобы удалить их из группы, по нажатию кнопки.

Далее на рисунке 2.10 представлен прототип страницы просмотра заявок на вступление в группу.

9 «А» класс

О группе Участники Заявки на вступление + Добавить занятие

ФИО участника	Почта
☐ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☐ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☒ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☐ Иванов Иван Иванович	ivanoff11247348@gmail.com
☐ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com
☒ Иванов Иван Иванович	ivanoff11247348@gmail.com
☒ Петров Ксенофонт Дормидонтович	ivanoff11247348@gmail.com

Отклонить Принять

Рисунок 2.10 – Прототип страницы управления заявками на вступление в группу от лица преподавателя

Здесь, администратор группы может выбрать одного или нескольких пользователей и принять их в группу, или отклонить им доступ в группу. Об этом, пользователи увидят в своем интерфейсе.

Далее на рисунке 2.11 представлена форма создания занятия внутри группы.

Создать занятия

Название
Например, Авиамоделирование

Дата
12 марта 2025

Время
16:00 - 16:45

☐ Весь день

Повтор занятия
Не повторяется

Описание
Напишите описание

Пригласить участников
example@mail.com

Сохранить

Рисунок 2.11 – Прототип страницы создания занятия в группе от лица преподавателя

Далее рассмотрим страницы управления группами с стороны обучающегося. На рисунке 2.12 представлена схожая таблица с группами.



В самом правом столбце таблицы представлена информативная кнопка, информирующая пользователя о том, является ли он участником группы и если нет, то может ли он вступить в неё сразу или лишь отправить запрос на вступление в закрытую группу.

47

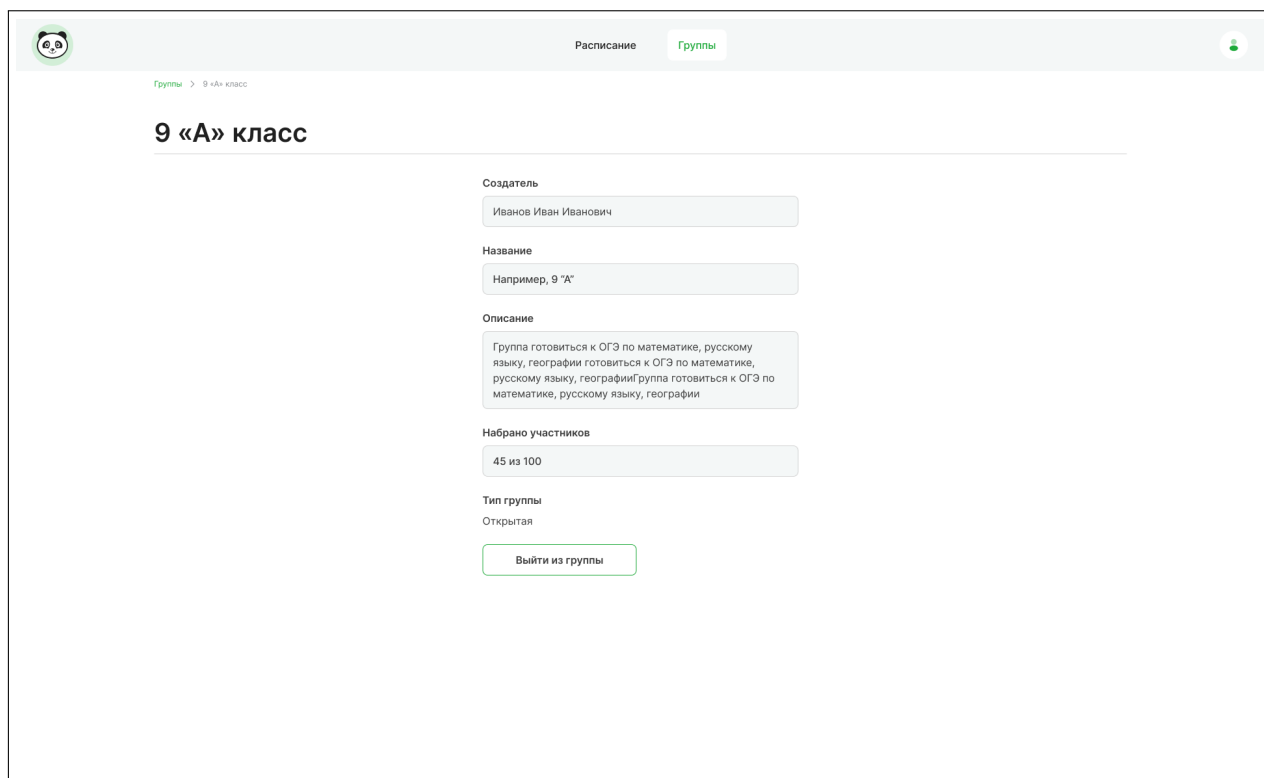


Рисунок 2.13 – Прототип страницы просмотра одной группы от лица обучающегося

Здесь пользователь имеет возможность посмотреть более подробную информацию о группе и кнопку покидания группы.

2.5 Проектирование и разработка клиентской части приложения

Прежде чем углубляться в детализированное рассмотрение ключевых аспектов к процессу проектирования и реализации пользовательского интерфейса и функциональных компонентов клиентской части веб-приложения, представляется крайне важным и методически обоснованным провести всесторонний анализ существующих на текущий момент времени технологических решений, инструментальных платформ и фреймворков, доступных разработчикам.

В современных условиях разработки простого использования нативного JavaScript [33] уже явно недостаточно для обеспечения требуемого уровня производительности, масштабируемости и удобства разработки. В связи с этим необходимо применение фреймворков, которые значительно упрощают процесс создания клиентской части приложений.

Сравнительная характеристика JavaScript "фреймворков" [34] представлена в таблице 2.2.

Таблица 2.2 - Сравнительный анализ JavaScript фреймворков

JavaScript фреймворк	Преимущества	Недостатки
Angular	Наибольшее сообщество и количество готовых решений TypeScript поддержка на уровне ядра Хорошая структура для больших enterprise-проектов	Сложная кривая обучения Требует дополнительных библиотек для полноценного стэка (роутинг, state-менеджмент) Менее гибкий по сравнению с React/Vue
React	Наибольшее сообщество и количество готовых решений Виртуальный DOM [35] для оптимизации рендеринга Гибкость архитектуры (можно собирать стэк под себя)	JSX может усложнять понимание для новичков Требует дополнительных библиотек для полноценного стэка (роутинг, state-менеджмент) Частые изменения best practices
Svelte	Нет виртуального DOM (компиляция в нативный JS) Простейший синтаксис и минимальный boilerplate Отличная производительность	Меньшее сообщество и экосистема Менее подходит для сложных state-зависимых приложений Проблемы с масштабированием в больших проектах
Vue	Плавная кривая обучения Гибкость: можно использовать как целиком фреймворк, так и частично Оптимальная производительность (комбинация Virtual DOM и реактивности)	Меньше корпоративных решений

Проанализировав все доступные варианты среди сред разработки, было принято решение остановиться на Vue.js из-за следующих преимуществ:

1. Простота интеграции: легко добавить в существующий проект и есть возможность постепенного внедрения.
2. Композиционный API предоставляет лучшую организацию кода по сравнению с Options API, а также проще выносить логику в много используемые функции.

3. Оптимальная производительность: новая реактивная система на Proxy и Tree-shaking из коробки.

4. Экосистема представлена Vue Router для навигации, Pinia/Vuex для state-менеджмента и Vue-CLI-Service для мгновенного hot-reload.

Архитектура клиентской части приложения сформирована с помощью наложения одних компонентов над другими. С этой схемой можно ознакомиться на рисунке 2.14.

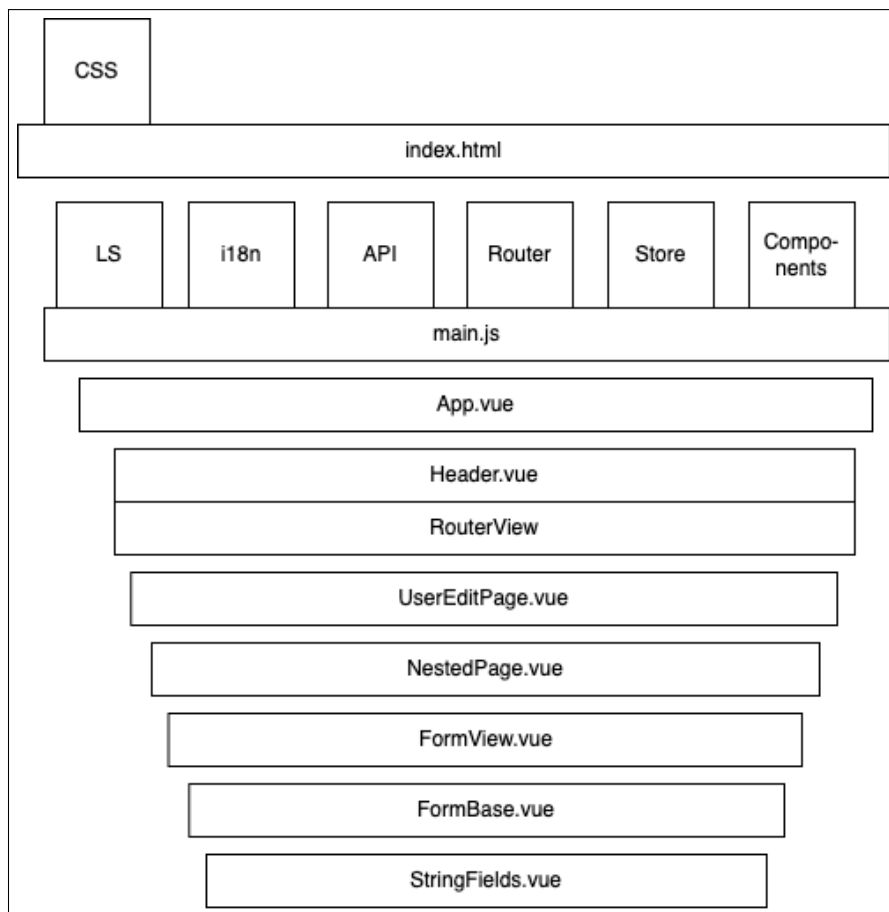


Рисунок 2.14 – Построение компонентов на основе страницы личного кабинета

Здесь точкой входа является файл index.html - к которому монтируется Vue js приложение, а также все CSS стили.

Монтирование самого приложения происходит в файле main.js. В нем регистрируются все используемые глобальные модули для работоспособности приложения:

1) LS (LocalStorage). Локальное хранилище данных в браузере. Обеспечивает хранение данных до тех пор, пока пользователь не очистит кеш страницы или не зайдет под инкогнито;

- 2) `Router`. Модуль позволяет реактивно изменять локаль приложения, переключаясь моментально между языками в интерфейсе;
- 3) `API`. По своей сути является объектом, который разделяет API для использования внутри клиентской части на смысловые части. Значениями же являются функции, которые посылают запросы к серверу;
- 4) `Router`. Зарегистрированный маршрутизатор, позволяющий монтировать определенные View компоненты к URI путям.
- 5) `Store`. Хранилище данных между различными страницами и компонентами. Позволяет хранить глобальные состояния и описания получаемых полей с сервера. Является слоем данных;
- 6) `Components`. Используемые компоненты, позволяющие не импортировать каждый файл при использовании компонента, а предоставив глобальный доступ к ним.

Все вышеперечисленные модули глобальны и доступны из всех Vue компонентов из-за интеграции их на уровне файла `main.js`.

`UserEditPage` - View компонент, который определяет наполнение и функционал этой страницы. В данном примере на странице редактирования мы видим форму `FormView`.

`FormView` - общий компонент просмотра, редактирования и создания сущностей. Предоставляет функционал сохранения, отмены и навигации между состояниями формы. С помощью API запрашивает данные о пользователе и отображает их. При изменении данных и нажатии на кнопку сохранить отправляет запрос на обновление данных о пользователе.

`FormBase` - основа для `FormView`. Используя `Store`, этот компонент определяет правила для отображения полей внутри формы, заполняет их значениями, а также валидирует их и собирает ошибки валидации при вводе значений в форму.

Далее, рассмотрим функциональность приложения с точки зрения навигации по страницам приложения.

Навигационная система [36] представляет собой комплексное функциональное решение, реализующее логику переходов между разделами веб-приложения. Данный механизм играет ключевую роль в организации пользовательского взаимодействия, обеспечивая не просто возможность перемещения между страницами, но и поддерживая целостное восприятие интерфейса.

В примере выше рассматривается страница редактирования данных о пользователе, поэтому имеем RouterView - vue компонент, предоставляющий функционал рендера компонента, основанного на текущей локации пользователя внутри приложения. Все существующие страницы внутри проекта описаны на рисунке 2.15.

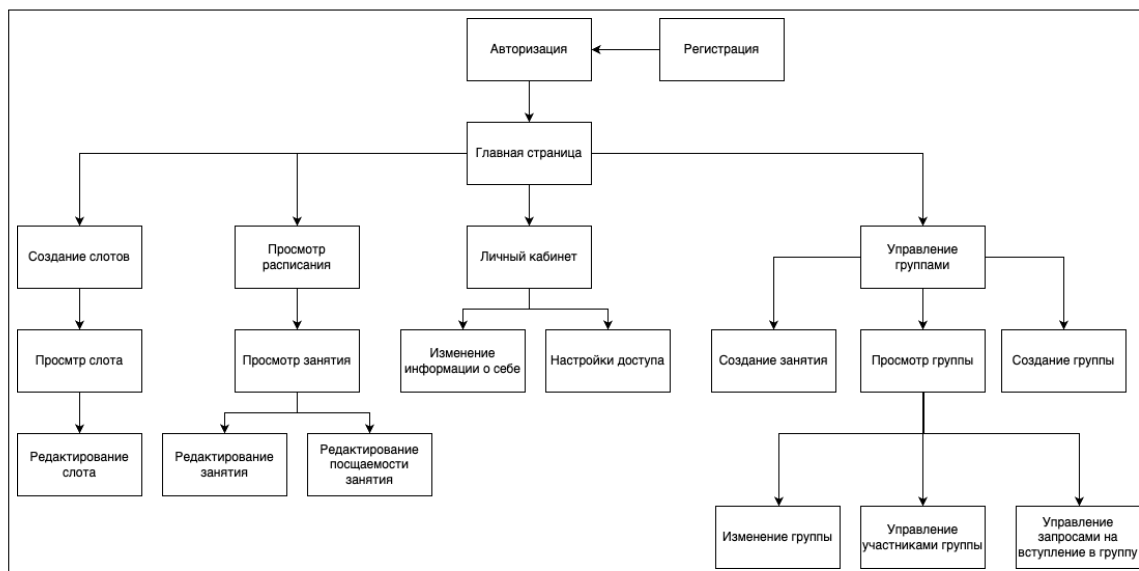


Рисунок 2.15 – Карта существующих страниц в приложении

Навигация по сайту происходит по разделам и под-разделам. К разделам относят:

1. Группы.
2. Слоты.
3. Пользователь.
4. Расписание.
5. Авторизация.

К подразделам относятся более подробные маршруты:

- 1) /show - просмотр, карточка;
- 2) /edit - редактирование;
- 3) /new - создание;
- 4) /list - просмотр списка;

Для реализации этого функционала была использована встроенная библиотека Vue-Router.

Для начала, были описаны все существующие маршруты в одной переменной sections. Представлена на листинге 2.8.

Листинг 2.8 — Все секции маршрутов в приложении

```
const sections = [
  { name: 'auth', actions: ['login', 'register'], useId: false },
  { name: 'home', actions: null, useId: false },
  { name: 'user', actions: ['show', 'edit'], useId: true },
  { name: 'groups', actions: ['list', 'show', 'edit', 'new'], useId: true },
  { name: 'events', actions: ['show', 'edit', 'new'], useId: true },
  { name: 'slots', actions: ['show', 'edit', 'new'], useId: true },
]
```

Здесь `useId` определяет использовать ли `id` в качестве части последней маршрута.

Далее, основываясь на переменной `sections`, я генерирую все маршруты для роутера внутри приложения с помощью функции `generateRoutes`, представленной на листинге В.3.

Здесь идет итерация по каждой секции переменной `sections` и выполняются следующие проверки.

Сначала вычисляется имя компонента - файла Vue который будет связан с каждым маршрутом. У меня все страницы хранятся в папке `/pages`. Выбирается поле `name` у каждой секции, первая буква делается заглавной и в конце добавляется слово `Page`. Таким образом секция с `name events` стала `EventsPage`

Далее этот компонент по умолчанию импортируется в переменную `defaultComponent`.

Далее, если у секции есть массив `actions` то по каждому действию импортируется новый компонент с следующей логикой. Например, если у секции `events` есть действие `list`, то импортируется компонент `EventsListPage` из папки `pages/events`. Иначе если действия не описаны, то используется компонент по умолчанию.

Далее, рассмотрим реализованный функционал взаимодействия клиентской части приложения с серверной путем создания интерфейса отправки запросов.

API (Application Programming Interface) [37] – это программный интерфейс, который позволяет клиентской части веб-приложения взаимодействовать с сервером по определенным правилам.

API построено по принципу единой точки входа с последующим роутингом на бэкенде. Основные компоненты:

- 1) `api.help.js` - низкоуровневые HTTP-утилиты;
- 2) `api/index.js` - фасад для работы с API;
- 3) Глобальный объект `api` - доступен через `app.config.globalProperties`.

Транспортный уровень. Обработка исходящих запросов с помощью `XMLHttpRequest` с дополнительной логикой обработки ответов представлена на листинге 2.9.

Листинг 2.9 — Обработка запросов

```
const callXhr = ({ url, method, headers, body, ...params }) =>
  new Promise(resolve => {
    const Xhr = new XMLHttpRequest()
    // request config
    Xhr.open(method, url)
    // handlers
    Xhr.send(body)
  })
```

Особенности:

1. Буферизация ответов для контроля размера данных.
2. Единая точка обработки ошибок.
3. Поддержка прогресса загрузки.

Используется цепочка обработчиков, представленная на листинге 2.10.

Листинг 2.10 — Цепочка обработки

```
const handleRequest = (promise, handlers) =>
  promise.then(res => {
    const handler = handlersByStatusCode[res.statusCode] || defaultHandler
    return handler(res)
  })
```

Где `handlersByStatusCode` содержит специфичные обработчики для каждого HTTP-статуса.

Доступен через Vue-приложение и содержит логически сгруппированные методы (листинг 2.11)

Листинг 2.11 — Структура API

```
const customApi = {
  auth: {
    login: makeApiFn('/auth/login'),
    register: makeApiFn('/auth/register')
  },
}
```

Продолжение листинга 2.11

```
groups: {  
  list: makeApiFn('/groups/list'),  
  create: makeApiFn('/groups/create')  
}
```

Особенности реализации:

- 1) автоматическая обработка ошибок: централизованная система через `handlersByStatusCode`;
- 2) безопасность: автоматическое добавление JWT-токена из `LocalStorage`;
- 3) гибкость: поддержка как `JSON`, так и `FormData`;
- 4) мониторинг: логирование всех запросов/ответов в консоли.

Пример вызова `api` метода из компонента `Vue` представлен на листинге 2.12.

Листинг 2.12 — Пример вызова

```
// In Vue component  
const { body, ok } = await this.$api.auth.login(credentials)  
if (ok) {  
  // ok call back  
} else {  
  // errors call back  
}
```

API становится реактивным [38] через интеграцию с `Vue` на листинге 2.13

Листинг 2.13 — Интеграция с Vue

```
export const api = new (function() {  
  this.install = (app) => {  
    app.config.globalProperties.$api = reactive(customApi)  
  }  
})()
```

Это позволяет:

- 1) использовать API в любом компоненте через `this.$api`;
- 2) получать реактивные обновления при изменении данных;
- 3) интегрировать с `Vue DevTools`.

Данная архитектура API обеспечивает единый стиль взаимодействия с серверной части приложения, централизованную обработку ошибок и про-

стую интеграцию с Vue-компонентами при сохранении гибкости для различных сценариев использования.

Далее, рассмотрим процессы авторизации в приложении.

Исходя из функциональных требований, реализовано 3 механизма авторизации: локальный, через Yandex OAuth и Google OAuth. Внешний вид финальной страницы авторизации представлен в полном виде на рисунке 2.16.

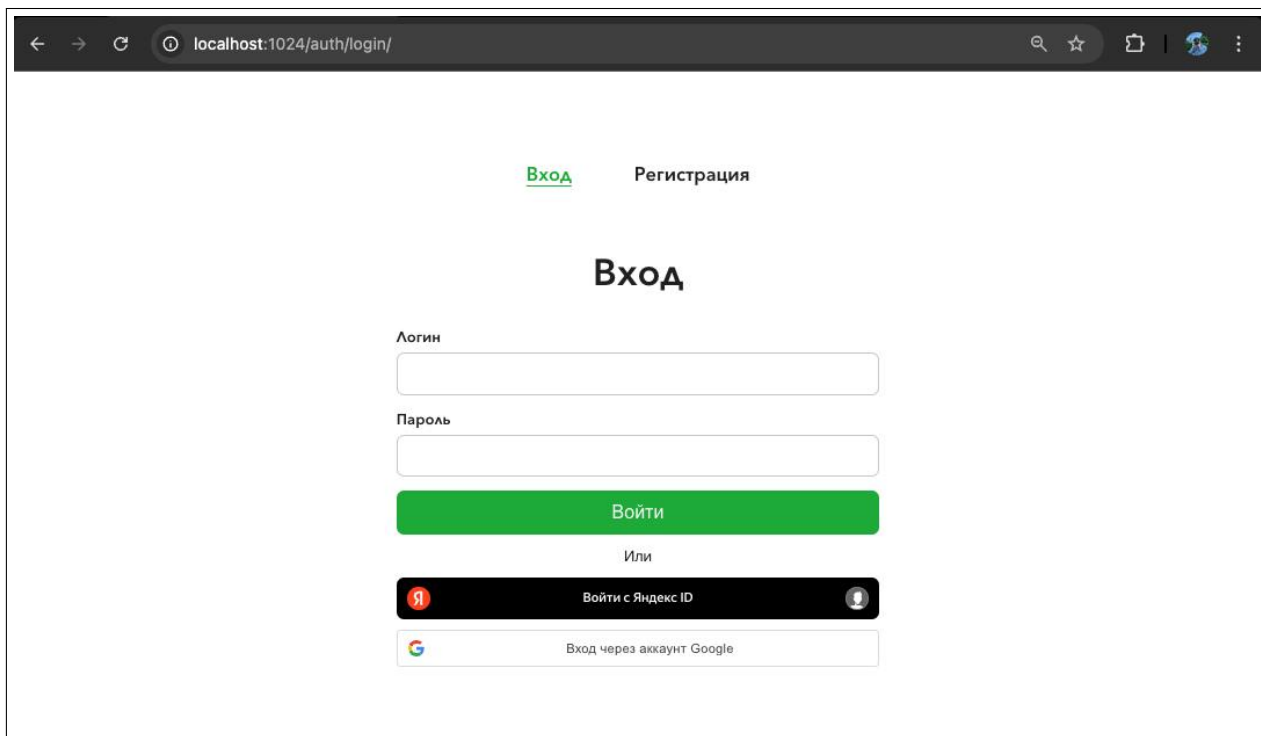


Рисунок 2.16 – Страница авторизации

Рассмотрим каждый из них по очереди.

Локальная авторизация - авторизация пользователей, которые сами зарегистрировались в системе с помощью внутренней формы регистрации, код которой представлен на листинге 2.14.

Листинг 2.14 — Форма авторизации

```
<FormView displayName="login" action="edit">
  <template #form-bottom="{ entity }">
    <div class="form-bottom-column">
      <BaseBtn @click="authorize(entity)"Войти></BaseBtn>
      Или
      <div id="auth-bottom"></div>
      <div id="auth-bottom2"></div>
    </div>
  </template>
</FormView>
```


Рассмотрим форму авторизации, представленную на листинге 2.14. На ней мы можем наблюдать компонент `FomView` у которого мы видимшь лишь 3 функциональные кнопки. Сама форма отображает всего 2 поля: поле ввода логина и поле ввода пароля с кнопками авторизации.

При нажатии на кнопку "Войти" срабатывает событие `@click` и вызывается функция `authorize` с параметром `entity`, полученный из формы. Реализация функции на листинге 2.15.

Листинг 2.15 — Метод авторизации пользователя

```
authorize(entity) {  
  this.$api.auth.login({ ...entity }, res => {  
    if (res?.detail) return  
    this.$ls.token = res.access_token  
    this.$ls.current_user = res.user_id  
    this.$store.commit('setUser', res.user_id)  
    this.$router.replace('/home')  
  })  
},
```

Здесь вызывается API метод авторизации `login`, которому в качестве параметров передаются все полученные значения из формы. Вторым параметром передается `callBack` функция, которая выполнится синхронно после успешного выполнения запроса. Она получается в качестве параметра `res` - результат, полученный с сервера.

С серверной части приходит следующее: JWT токен доступа в приложение. Без него доступ невозможен. Также получаем уникальный идентификатор пользователя, для последующих запросов на получение информации о нем в других страницах. Эти данные мы кладем в локальное хранилище браузера, а также в наше внутреннее хранилище, чтобы было удобно доставать его при надобности. После чего происходит перенаправление пользователя на главную страницу.

Авторизация через внешний сервис Яндекс. Для начала необходимо зарегистрировать приложение внутри системы авторизации яндекса. Сделать можно это на их сайте [39].

При регистрации приложения необходимо указать список данных, которые мы хотим получать от авторизуемого пользователя, а также URL страницы на которую Яндекс будет перенаправлять пользователя после успешной авторизации (рисунок 2.17).

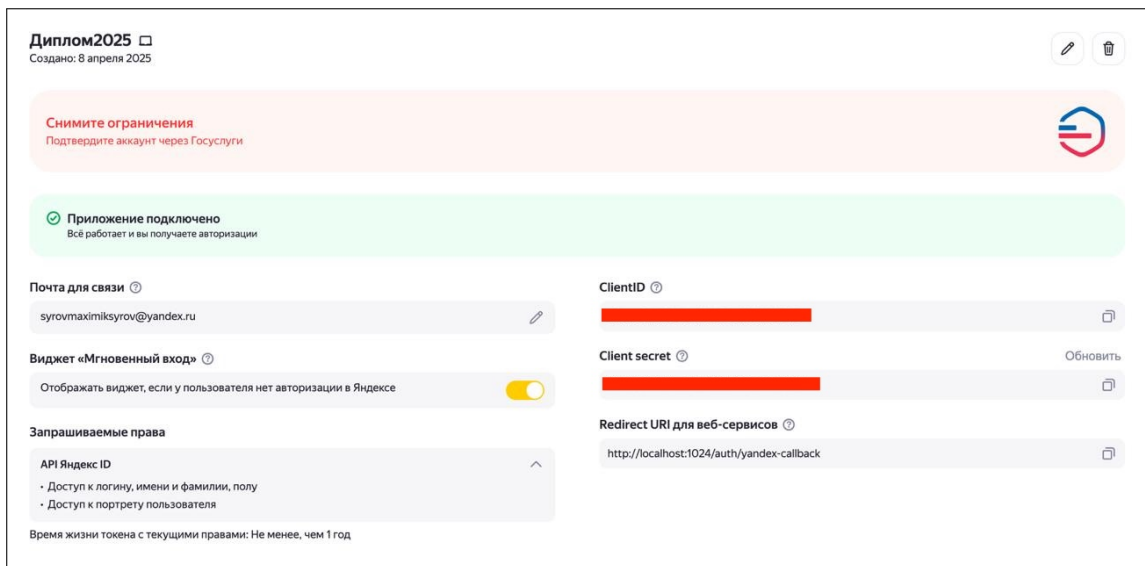


Рисунок 2.17 – Страница с доступом приложения в Яндекс OAuth

Теперь необходимо полученный ClientID сохранить в проекте и использовать далее.

Механизм получения данных пользователя через Яндекс:

1. Подключить кнопку авторизации в приложение (листинг В.4) при создании страницы авторизации. Здесь подключается скрипт, предоставленный Яндексом, который добавляет в мое DOM дерево свой компонент.
2. Пользователь нажимает на неё, авторизуясь через свой аккаунт.
3. С моей страницы на сервер Яндекс отправляется запрос на получение токена доступа с помощью моего ClientID (листинг В.4, строки 7-13).
4. После успешной авторизации пользователя, Яндекс перенаправляет пользователя на страницу, указанную в Личном кабинете авторизации и в запросе на получение токена (они обязательно должны совпадать) (листинг В.4, строка 12).
5. Поскольку Яндекс открывает всплывающее окно, а не перенаправляет уже на открытую вкладку, получить токен на той же странице нельзя. Поэтому указана страница `auth/yandex-callback`, которая, получив токен доступа (листинг 2.16, строки 2 - 5), запрашивает у Яндекса информацию о пользователе и сохраняет её в `LocalStorage` (листинг 2.16, строки 7, 8). После чего закрывает всплывающее окно (листинг 2.16, строка 9).
6. На старой странице авторизации идет подписка на изменения в `LocalStorage` под ключом `yandex_user_data` (листинг 2.17). Таким образом, при изменении значения под этим ключом срабатывает функция, которая по-

лучает из LocalStorage данные, расшифровывает их и выполняет последовательно процесс регистрации, а затем авторизации. В случае если пользователь уже зарегистрирован через этот аккаунт, пользователь авторизуется автоматически.

Листинг 2.16 — Получение данных о пользователе из Яндекса

```
1 mounted() {
2   const url = new URL(window.location.href)
3   const hash = url.hash.substring(1)
4   const params = new URLSearchParams(hash)
5   const accessToken = params.get('access_token')
6   if (!accessToken) return
7   this.$api.auth.yandexGetData(accessToken, 'json', data => {
8     this.$ls.setItemEncrypt('yandex_user_data', JSON.stringify(data))
9     window.close()
10  })
11 },
```

Листинг 2.17 — Регистрация и авторизация с данными полученные из Яндекса

```
1 this.$ls.subscribe('yandex_user_data', () => {
2   const userData = JSON.parse(this.$ls.getItemDecrypt('yandex_user_data'))
3   this.$api.auth.register(
4     {
5       name: userData.first_name,
6     },
7     () => {
8       this.authorize({
9         login: userData.login,
10        password: password,
11      })
12    }
13  )
14 })
```

Авторизация через внешний сервис Google.

По аналогии с сервисом Яндекса, в Google OAuth необходимо зарегистрировать веб-приложение в их сервисе для управления авторизаций [40] для получения данных о пользователях (рисунок 2.18).

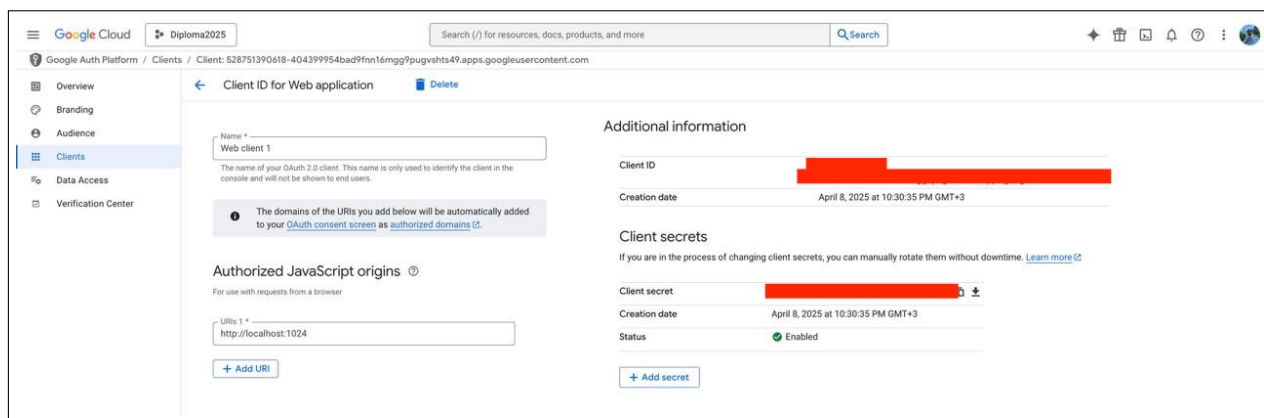


Рисунок 2.18 – Страница с доступом приложения в Google Oauth

Процесс авторизации пользователя через Google Oauth схож с методом Яндексa, но несколько проще.

Механизм получения данных пользователя через Google:

1. Подключить кнопку авторизации в приложение (листинг В.5) при создании страницы авторизации. Здесь подключается скрипт, предоставленный google, который добавляет в мое DOM дерево свой компонент (листинг В.5, строка 2).
2. Пользователь нажимает на неё, авторизуясь через свой аккаунт.
3. После авторизации пользователем в сплывающем окне, отправляется запрос к серверам Google в котором указывается (листинг В.5, строки 6-24): `client_id` - полученный ClientID при регистрации приложения, `response_type` - то, что мы ожидаем в результате запроса - `token`, те токен авторизации пользователя, `scope` - это политики доступа, перечисленные через пробел - те данные с которыми авторизуемый пользователь сможет поделиться для приложения, `callback` - функция которая отработает в случае успешного запроса к Google.
4. В случае успешного выполнения запроса, выполняется `callback` функция. В ней, одним из параметром является зашифрованный с помощью JWT - токен в `credentials` (листинг В.5, строки 7, 8), расшифровав который, можно получить данные пользователя.
5. С помощью полученный данных пользователя мы также его регистрируем и авторизуем с помощью описанных ранее методов (листинг В.5, строки 12 - 24).

2.6 Выводы по проектной части

В второй главе подробно рассмотрена реализация веб приложения. Основное внимание уделено технической реализации серверной части, клиентской части, а также клиентского интерфейса. Разработка велась параллельно и одновременно для минимизации ошибок при стыковке серверной и клиентской части.

Серверная часть реализована с использованием Python и фреймворка FastAPI с использованием POSTful API взаимодействия с клиентом. Описано движение данных от клиента к серверу, к классам валидации и к бизнес логике. Описаны алгоритмы авторизации а также взаимодействия с реляционной базой данных, которой было уделено особое внимание - структуре данных, а также специальных запросов на получение информации об обучающихся и их занятий. Установлены все связи между таблицами, обеспечивающие корректность выборок и поддержку фильтрации.

Серверная часть реализована с помощью Vue.js - фреймворка над JavaScript. Описана структура клиентской части, описаны созданные моудли, такие как маршрутизатор путей внутри приложения, API для отправки запросов на сервер, а также использование компонентного подхода внутри веб приложения.

В ходе второй главы была выполнена реализация всех ключевых компонентов веб-приложения. Система обеспечивает корректную работу пользовательского интерфейса, надежное взаимодействие с сервером и базой данных, поддержку сложных сценариев использования приложения для администрирования групповых занятий. Полученный результат является функционально завершенным и готовым к интеграции с дополнительными модулями.

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Развертывание приложения

Развертывание приложения происходит в 2 этапа. Первый этап - подготовка финальной сборки приложения к выпускаемой версии. И второй этап - настройка доменного окружения и запуск приложения в сети интернет

Для успешного запуска эксплуатационной версии как клиентской, так и серверной части, а также базы данных независимо друг от друга, я использую Docker [41].

Docker представляет собой платформу для контейнеризации приложений, обеспечивающую их изоляцию и воспроизводимость в различных средах выполнения. В рамках данной технологии ключевыми концепциями являются контейнеры, образы (images) и оркестрация, которые совместно решают проблему несоответствия сред разработки, тестирования и эксплуатации.

Контейнеры — это легковесные исполняемые среды, изолированные на уровне операционной системы. В отличие от виртуальных машин, они используют общее ядро ОС, что обеспечивает значительное снижение накладных расходов при запуске. Каждый контейнер инкапсулирует приложение со всеми его зависимостями (библиотеками, системными утилитами, конфигурациями), гарантируя идентичное поведение на любой платформе, поддерживающей Docker.

Образы служат шаблонами для создания контейнеров. Они формируются посредством многослойной сборки, где каждый слой соответствует конкретной инструкции в Dockerfile (например, установка пакетов, копирование файлов). Такой подход обеспечивает эффективное кэширование и повторное использование компонентов. Образы хранятся в реестрах, что упрощает их распространение и версионирование.

Практическая значимость Docker проявляется в следующих аспектах:

1. Унификация сред: устранение расхождений между окружениями разработчиков, CI/CD-серверов и production-серверов.
2. Масштабируемость: быстрое развертывание идентичных экземпляров приложения для обработки пиковых нагрузок.

3. Микросервисная архитектура: изолированное функционирование компонентов системы с индивидуальными требованиями к окружению.

Для начала необходимо подготовить серверную часть сервиса.

В серверной части фигурирует 2 докер контейнера: один для запуска базы данных, другой для содержания всей бизнес логики сервиса.

За это отвечает Dockerfile, представленный в листинге 3.1:

Листинг 3.1 — Dockerfile серверной части

```
FROM python:3.12
WORKDIR /app
COPY Pipfile Pipfile.lock /app/
COPY *.env /app/
RUN pip install pipenv
RUN pipenv install --ignore-pipfile
COPY ./app /app/
EXPOSE 8000
CMD ["pipenv", "run", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Здесь, сначала, на вход подается минимально допустимая версия питона, без которой серверная часть приложения не сможет скомпилироваться.

Далее переключаемся в рабочую директорию app внутри докер приложения. В этой директории располагаются все файлы, которым будет предоставлен ограниченный доступ через интернет.

Далее в директорию копируются файлы Pipfile и Pipfile.lock - они описывают все необходимые библиотеки и зависимости для нормального функционирования серверной части приложения.

Далее происходит установка всех библиотек зависимостей. И только потом с помощью демона uvicorn происходит запуск API на текущем хосте (локальном хосте) на 8000 порту. Теперь если пытаться обращаться к 8000 порту, то можно отправлять запросы к сервису.

Теперь рассмотрим запуск клиентской части приложения.

Поскольку сервис запускается в продакшн версии, использовать Node JS и встроенные скрипты для запуска приложения (npm run serve и прочее) не является возможным, поскольку такой способ подходит только для запуска в режиме разработки [42]. Соответственно, для функциональной работы мне понадобился самый простой функциональный сервис для запуска веб-сервера Nginx [43].

Nginx – это высокопроизводительный веб-сервер с открытым исходным кодом, который также может использоваться в качестве обратного прокси, кэширующего сервера и балансировщика нагрузки.

Конфигурация nginx представлена в файле nginx.conf и представлена на листинге 3.2.

Листинг 3.2 — Конфигурация Nginx

```
server {
    listen 1024;
    server_name localhost;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
        server_name m.syrov.ru www.m-syrov.ru;
        add_header Cache-Control "no-cache, no-store, must-revalidate";
        expires 0;
    }
    location /diploma {
        proxy_pass http://89.104.71.146:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Здесь Nginx выполняет простую функцию, позволяет получить доступ к статичным файлам моего веб приложения через обращение к 1024 порту.

Кроме этого, nginx проксирует все исходящие запросы клиентской части приложения от 1024 порта к 8000 порту, на котором запущено API, серверная часть приложения.

Для запуска nginx на виртуальной машине также понадобится docker с Docker файл с конфигурациями, представленные на листинге 3.3.

Листинг 3.3 — Dockerfile клиентской части

```
FROM node:18 as builder

WORKDIR /app
COPY . .
FROM nginx:alpine as production
COPY --from=builder /app/dist /usr/share/nginx/html
COPY --from=builder /app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 1024
```


Здесь, происходит слудующее: docker копирует содержимое папки frontend к себе в общую папку app, откуда выполняет копирование nginx файлов конфигурации и выставления 1024 порта в общий доступ.

Предварительно, я сделал build версию своего приложения через команду npm run build. Она использует встроенные методы Node js для оптимизации Vue файлов и компиляции их в нативный JavaScript код. Это нужно для из-за того, что браузеры умеют распознавать только нативный JavaScript код. При разработке браузер тоже читает нативный код [44], но ему предоставляются так называемые map карты по которым браузер может воссоздать первоначальный вид файлов до компиляции, предоставив разработчику средства для отладки кода)

Теперь, когда все докерфайлы готовы, можно собрать их и запустить их. За процесс сборки и запуска докер контейнеров в image файлы отвечает скрипт docker-compose.yml, представленный на листинге 3.4.

Листинг 3.4 — Скрипт docker-compose.yml

```
version: '3.8'
services:
  db:
    image: postgres:latest
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    image: backend:latest
    restart: always
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    depends_on:
      - db

  frontend:
    build: ./frontend
```

Продолжение листинга 3.4

```
ports:
  - "1024:1024"
depends_on:
  - backend
  - db
volumes:
  postgres_data:
```

Здесь идет полное описание всех докер контейнеров: db отвечает за базу данных, backend за серверную часть и frontend за клиентскую часть.

Теперь при запуске файла через команду ``docker compose up --build`` будут созданы и скомпилированы мои 3 контейнера и доступ к клиентской части приложения будет на 1024 порту.

Далее, рассмотрим процесс развертывания приложения в интернете.

Для публикации сервера в интернете, я использовал VPS [45].

VPS (Virtual Private Server) — это виртуальный выделенный сервер, представляющий собой изолированную среду на физическом сервере с гарантированными вычислительными ресурсами (CPU, RAM, дисковое пространство). В отличие от shared-хостинга, где ресурсы распределяются между множеством пользователей, VPS обеспечивает полный контроль над окружением за счет:

1. Виртуализации на уровне ОС или аппаратной виртуализации, что позволяет устанавливать произвольное ПО и настраивать параметры ядра.
2. Выделенных ресурсов, независимых от других пользователей сервера.
3. Root-доступа, обеспечивающего администрирование на уровне суперпользователя (установка серверного ПО, настройка брандмауэров, мониторинг).

Виртуальные сервера позволяют запустить на них веб приложение, подключить к ним доменное имя [46] и иметь доступ к веб приложению через сеть интернет.

Проанализировав рынок поставщиков доменных услуг [47], я остановился на рег.ру. Здесь я смог приобрести доменное имя m-sugov.ru на год за 130 рублей. Также, рег.ру предоставляет возможность приобрести VPS сервер за 540 рублей в месяц.

Получив доступы к своему VPS серверу и подключившись к нему по SSH, я перенес свои файлы проекта, запустил docker-compose, описанный выше, и открыл свой сервис в открытый доступ[48].

3.2 Проведение тестирования сервиса

Для проведения успешного тестирования созданного сервиса были созданы следующие сценарии тестирования и тест кейсы.

Тест-кейс [49] – это документированный набор шагов, условий и ожидаемых результатов, предназначенный для проверки конкретной функциональности или сценария работы программного обеспечения. Тест-кейс помогает систематизировать процесс тестирования и обеспечивает воспроизводимость проверок.

Сценарное тестирование [50] – это метод тестирования, при котором проверяется работоспособность системы в условиях, максимально приближенных к реальному использованию. Оно основывается на пользовательских сценариях (например, "регистрация нового ученика и запись его на занятие") и оценивает, как приложение ведет себя в комплексных ситуациях, а не только при изолированных проверках отдельных функций.

Тест создания открытой группы.

Шаги воспроизведения:

1. Зарегистрировать нового пользователя с ролью учителя и авторизоваться.
2. Нажать на кнопку создание группы.
3. Ввести правильные данные в поля формы.
4. Нажать кнопку "Сохранить".

Ожидаемым результатом теста является создание новой группы и событие редиректа на карточку созданной группы.

Фактический результат - тест выполнен.

Тест вступления в группу и авторизации через сторонний сервис.

Шаги воспроизведения:

1. Авторизоваться через Яндекс.
2. Открыть список групп.
3. Выбрать фильтр по чужим открытым группам.
4. Выбрать группу и нажать кнопку "Вступить".

Ожидаемый результат:

1. В карточке группы на вкладке участников есть имя авторизованного пользователя.
2. В расписании есть занятия группы, в которую вступил пользователь.

Фактический результат - тест выполнен.

Тест принятия пользователя в закрытую группу.

Шаги воспроизведения:

1. Авторизоваться за пользователя с ролью обучающегося.
2. Перейти в список групп и выбрать фильтр по закрытым группам.
3. Выбрать группу и нажать кнопку "Отправить запрос".
4. Выйти из аккаунта.
5. Авторизоваться за пользователя с ролью преподавателя.
6. Перейти в карточку своей закрытой группы.
7. В списке запросов на вступление выбрать заявку от ученика.
8. Нажать кнопку "Принять".

Ожидаемым результатом теста является:

1. Ученик должен перенестись в список участников группы.
2. Ученик должен увидеть себя в списке участников этой закрытой группы.

Фактический результат - тест выполнен.

Тест создания занятий без привязки к группе.

Шаги воспроизведения:

1. Авторизоваться за пользователя с ролью преподавателя.
2. Перейти в расписание.
3. Нажать кнопку "Добавить слот".
4. Заполнить правильные данные в форме, сохранить.

Ожидаемым результатом теста является видимость слота в расписании преподавателя, а также в разделе мои слоты в карточке пользователя.

Фактический результат - тест выполнен.

Тест изменения успеваемости обучающихся в слоте.

Шаги воспроизведения:

1. Авторизоваться за пользователя с ролью преподавателя.

2. Перейти в расписание.
 3. Нажать кнопку "Добавить слот".
 4. Заполнить правильные данные в форме, сохранить.
 5. Авторизоваться за пользователя с ролью обучающегося.
 6. Перейти в расписание, открыть расписание слотов преподавателя путем поиска его слотов в фильтре расписания.
 7. Найти слот и забронировать место в нем.
 8. Авторизоваться за пользователя с ролью преподавателя.
 9. Открыть карточку слота, перейти в раздел участников.
 10. Нажать на галочку в столбце успеваемость обучаемого.
- Ожидаемый результат теста является сохранение успеваемости при обновлении страницы.

Фактический результат - тест выполнен.

3.3 Анализ возможностей развития приложения

Работа над веб приложением продолжается. Развитие приложения предполагается проводить по нескольким направлениям: функциональность, производительность, внешний вид, монетизируемость.

С точки зрения нового функционала, основной упор будет сделан на достижение большего удобства пользователя и увеличения продуктивности во время использования приложения.

Самое приоритетное исправление - поиск и своевременное исправление возникающих ошибок отображения форм, таблиц и фильтров. Для этого планируется создание окружения автоматического компонентного, а также сценарного тестирования.

Следующим по приоритету стоит новый функционал. В будущих обновлениях планируется добавление возможности добавлять изображения к группам, а также просматривать группы в виде плиток. Это уменьшит когнитивную нагрузку пользователя и заставит меньше вчитываться в описания групп и позволит добавить большей индивидуальности группам.

Далее планируется функционал синхронизации расписания с встроенными календарями мобильных устройств на платформах Android и iOS. Это позволит пользователям не терять свои предстоящие занятия.

Следующей приоритетной задачей будет добавление разделов аналитики и статистики. Необходимо будет дать информацию преподавателям об успеваемости по каждой группе, а также статистические данные по заработкам за предыдущие периоды, а также формирование тренда на будущий заработок. Для обучающихся же, разделы статистики будут полезны для понимания своей успеваемости как по отдельным предметам, так и за отдельные периоды. Это позволит пользователям лучше понимать свое расписание и вносить корректировки в своё обучение.

Для повышения производительности приложения, первым делом, необходимо будет расширить вычислительные возможности VPS сервера, на котором развернуто веб приложение. Планируемое улучшение сервиса должно включать в себя как минимум 16Гб оперативной памяти, 4-х ядерный процессор с средней частотой 2.5Ггц, а также порядка 500Гб встроенной памяти.

Данное расширение позволит увеличить пропускную способность сервиса с 100 до 1000 пользоателей, одновременно использующих сайт.

Также, для увеличения производительности, следует выделить время на проведение рефакторинга отдельных модулей и функций для уменьшения комплексности алгоритмов, а также ускорению работы сервиса.

Следующим важным шагом для увеличения производительности станет добавление нового типа базы данных Redis [51]. Redis позволяет хранить данные не на жестком диске виртуальной машины, а напрямую в оперативной памяти. Это позволит ускорить самые частые запросы к базе данных и сократить время ожидания пользователей под нагрузкой. Однако redis имеет ряд недостатков. Во первых, из-за особенности работы оперативной памяти, при выключении работы сервиса, все данные в оперативной памяти будут очищены. Также, оперативная память существенно меньше чем память жесткого диска, поэтому она будет полезна только для постоянно изменяемых данных, долгое хранение которых не имеет смысла.

Redis будет идеален для временного хранения истекших токенов доступа пользователей. Поскольку время жизни каждого токена доступа составляет всего 30 минут по умолчанию, ежедневное очищение этой таблицы не приведет к потере каких-либо важных данных, а только ускорит работу сервиса, потому что при каждом запросе к любому эндпоинту происходит как минимум 1 обращение к этой таблице.

Для поддержания актуальности внешнего вида веб-сайта будет продолжена работа над улучшением и изменениями стилей и внешнего вида различных компонентов внутри приложения.

Самым крупным из таких обновлений будет продолжена работа над созданием адаптивной версии веб сайта под устройства с узким экраном, а в дальнейшем полный адаптив под мобильные устройства. Это улучшение позволит пользователям открывать приложение не только с домашних компьютерных устройств, но и с мобильных.

Для финансирования всех вышеперечисленных нововведений, необходима частичная монетизация сервиса. Планируется, что основной функционал останется бесплатным, однако, часть функционала, который заточен на использование основных мощностей и большого количества пользователей - будет платным.

Так, планируется: В платном плане - создание неограниченного количества групп (фактически до 250 групп), в бесплатном плане до 30 групп.

Максимальное количество участников одной группы 30 человек - в бесплатном плане, до 150 человек в платном плане.

Максимальное количество участников внутри занятия будет также ограничено 30 пользователями в бесплатном плане и 150 в платном.

Также, размещение рекламы. В бесплатном плане планируется добавить по одному рекламному баннеру на каждую старницу приложения. В платном плане реклама отображаться не будет.

Итак, расчет стоимости плана будет зависеть от среднего количества пользователей постоянно использующих сервис, а также от стоимости одного просмотра и клика по рекламному баннеру.

Для расчета средней стоимости месячной подписки сервиса можно использовать следующую усовершенствованную формулу, которая учитывает как затраты, так и потенциальный доход от рекламы:

$$S = (M + F) - (A \times C \times (V \times P_v + K \times P_k)), \quad (1)$$

где S - итоговая стоимость подписки для пользователя (руб/мес);

M - затраты на хостинг и VPS (руб/мес);

F - фиксированная зарплата разработчику;

A - среднее количество показов рекламы на пользователя в день;
 C - количество активных пользователей;
 V - CTR (click-through rate) - процент кликов по рекламе (например, 0.02 для 2%);
 P_v - доход за 1000 показов (RPM) в рублях;
 K - конверсия в клики (например, 0.5% = 0.005);
 P_k - доход за клик в рублях.

С помощью формулы (1) приведем предварительные расчеты.

Предположим, что постоянные затраты $(M, F) = 50,000$ руб/мес и имеем 1,000 активных пользователей (C).

Также, условно считаем, что имеем 5 показов рекламы на пользователя в день (A).

30 дней в месяце, то есть $5 \times 30 = 150$ показов на одного пользователя в месяц. Следовательно:

RPM (P_v) = 50 руб за 1000 показов, те 0.05 руб за показ.

CTR (V) = 2% те 0.02.

Доход за клик (P_k) = 5 (руб.)

Расчет дохода от рекламы:

$$\begin{aligned}
 S &= C \times (A \times 30) \times (P_v + V \times P_k) = \\
 &= 1,000 \times 150 \times (0.05 + 0.10) = \\
 &= 1,000 \times 150 \times 0.15 = 22,500 \text{ (руб.)}
 \end{aligned} \tag{2}$$

Итоговая стоимость подписки:

$S = 50,000 - 22,500 = 27,500$ (руб.)

Распределение на пользователей:

Если 10 % пользователей платные: 100 платных пользователей.

$27,500 / 100 = 275$ (руб/мес) за одну подписку.

3.4 Выводы по внедрению и эксплуатации

В результате третьей главы достингута цель по развертыванию приложению, описаны схемы запуска для подготовки приложения к запуску в глобальной сети интернет. В результате 3 главы имеем готовое веб приложение, размещенное на своём доменном имени в глобальной сети интернет.

Также особое внимание было уделено тестированию ключевых сценариев использования веб приложения, в результате которого была достигнута завершенность проекта как с функциональной точки зрения, так и с технической точки зрения.

Помимо завершенного веб приложения были рассмотрены перспективы дальнейшего развития приложения с функциональной, производительной, визуальной а также денежной точек зрения. Продумана стратегия дальнейшего развития приложения в будущем.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана специализированная веб-платформа для администрирования групповых занятий, направленная на решение ключевых проблем частных преподавателей и малых образовательных проектов. В рамках исследования проведен анализ предметной области, целевой аудитории и конкурентных решений, что позволило сформулировать требования к системе и спроектировать оптимальную архитектуру приложения.

Реализация платформы включала разработку серверной части на Python (FastAPI) с RESTful API, клиентского интерфейса на Vue.js и реляционной базы данных с продуманной структурой для эффективного управления учебными процессами. Тестирование подтвердило корректность работы всех модулей, включая автоматизацию расписаний, учет посещаемости и платежей, а также управление группами обучающихся.

Дальнейшее развитие проекта предполагает:

1. Расширение функционала: интеграция с платежными системами, добавление мобильного приложения, внедрение аналитики успеваемости.
2. Оптимизацию производительности: масштабирование серверной части, кэширование данных, улучшение отзывчивости интерфейса.
3. Коммерциализацию: запуск пилотного внедрения в локальных образовательных проектах, разработка монетизации (подписка, премиум-функции).
4. Масштабирование: адаптацию платформы для корпоративного обучения и дополнительных рынков.

Полученные результаты демонстрируют готовность решения к практическому применению, а намеченные направления развития открывают перспективы для превращения проекта в полноценный коммерческий продукт.

СПИСОК ЛИТЕРАТУРЫ

1. Карлов, И. А. Анализ цифровых образовательных ресурсов и сервисов для организации учебного процесса школ / И. А. Карлов и др. ; Нац. исслед. ун-т «Высшая школа экономики», Ин-т образования. – М. : НИУ ВШЭ, 2020. – 72 с. – ISSN 2500-0608. – URL: <https://clck.ru/3MEihT> (дата обращения: 02.01.2025).
2. Коблова, М. В. Использование электронной системы обучения в образовательном процессе / М. В. Коблова // Компьютерные науки и информационные технологии : материалы Междунар. науч. конф., Саратов, 01–04 июля 2020 г. / отв. ред. В. А. Твердохлебов ; отв. за вып. Т. В. Семенова, А. Г. Федорова. – Саратов : ИЦ “Наука”, 2020. – С. 139–141.
3. Саидова, К. З. Анализ деятельности репетиторов / К. З. Саидова, Г. Р. Пожидаев, И. Д. Котилевец, И. А. Иванова // Информатика и образование. – 2021. – Т. 36, № 6. – С. 18–28. – ISSN 0234-0453. – URL: <https://clck.ru/3MEijT> (дата обращения: 08.02.2025).
4. Баланов, А. Н. Прототипирование и разработка пользовательского интерфейса: оптимизация UX : учеб. пособие для вузов / А. Н. Баланов. – СПб. : Лань, 2024. – 220 с. – ISBN 978-5-507-49211-4. – URL: <https://e.lanbook.com/book/414929> (дата обращения: 21.02.2025).
5. Клименко, И. С. Управление в организационных системах: учебное пособие для вузов / И. С. Клименко, Е. А. Палкин. — 2-е изд., стер. — Санкт-Петербург: Лань, 2025. — 324 с. — ISBN 978-5-507-52214-9. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://clck.ru/3MEiku> (дата обращения: 15.02.2025).
6. ЯРепетитор [Электронный ресурс]. — URL: <https://web.yarepetitor.com/auth> (дата обращения: 16.02.2025).
7. Гутгарц, Р. Д. О формализации функциональных требований в проектах по созданию информационных систем / Р. Д. Гутгарц, Е. И. Провилков // Программные продукты и системы. – 2019. – № 3. – С. 349–357. – EDN GXHPFX. – URL: <https://elibrary.ru/item.asp?id=41376084> (дата обращения: 15.02.2025).
8. Петрова, И. Р. Методология функционального моделирования IDEF0 / И. Р. Петрова, Р. Х. Фахртдинов, А. А. Сулейманова. – Казань : Казан. Ун-т,

2018. – 68 с. – URL: https://kpfu.ru/staff_files/F1506701798/Metodichka_IDEF.pdf (дата обращения: 23.02.2025).

9. Работа веб-приложения: особенности, отличия от сайта и виды [Электронный ресурс]. – URL: <https://skyeng.ru/magazine/wiki/it-industriya/chto-takoe-veb-prilozhenie/> (дата обращения: 16.02.2025).

10. Основы HTML [Электронный ресурс]. – URL: <https://clck.ru/3MEixK> (дата обращения: 20.02.2025).

11. Макфарланд, Д. С. Новая большая книга CSS = CSS: The Missing Manual. – 1-е изд. – СПб. : Питер, 2016. – 720 с. – ISBN 978-5-496-02080-0.

12. ECMAScript® 2025 Language Specification [Электронный ресурс]. – URL: <https://tc39.es/ecma262/> (дата обращения: 01.03.2025).

13. Method Definitions [Электронный ресурс]. – URL: <https://clck.ru/3MEizV> (дата обращения: 02.03.2025).

14. Часто Задаваемые Вопросы [Электронный ресурс]. – URL: <https://clck.ru/3MEj3k> (дата обращения: 04.03.2025).

15. SQL/JSON standard-2016 conformance for PostgreSQL, Oracle, SQL Server and MySQL [Электронный ресурс]. – URL: <https://obartunov.livejournal.com/200076.html> (дата обращения: 04.03.2025).

16. Урванова, Е. С. Какую систему управления базами данных с открытым исходным кодом выбрать для проекта / Е. С. Урванова // Студенческий. – 2024. – № 21-2(275). – С. 47-49. — Текст: электронный // Elibrary: электронно-библиотечная система. — URL: <https://www.elibrary.ru/item.asp?id=67863155> (дата обращения: 10.03.2025).

17. Silberschatz, A., Korth, H., Sudarshan, S. Database System Concepts. – 4th ed. – New York : McGraw-Hill, 2002. – Section 4.10.2: Join Types and Conditions. – p. 166. – ISBN 0072283637.

18. TIOBE Index for April 2025 [Электронный ресурс]. – URL: <https://www.tiobe.com/tiobe-index/> (дата обращения: 01.04.2025).

19. FastAPI Documentation [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 01.04.2025).

20. MVC XEROX PARC 1978–79 [Электронный ресурс]. – URL: <https://wayback.archive-it.org/10370/20180425071111/http://folk.uio.no/trygver/themes/mvc/mvc-index.html> (дата обращения: 05.04.2025).

21. RFC 7231 — Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content / Fielding, R., Reschke, J. [Электронный ресурс]. – URL: <https://clck.ru/3MEj6C> (дата обращения: 06.04.2025).
22. Martin, J. Managing the Data-base Environment. – Englewood Cliffs, N.J. : Prentice-Hall, 2010. – 381 p. – ISBN 0-135-50582-8.
23. Особенности формирования моделей и построение базы данных в бизнесе на платформе Django ORM / М. Б. Тлебаев и др. // Вестник Кыргыз. Гос. Ун-та им. И. Арабаева. – 2023. – № 2. – С. 299–309. – DOI 10.33514/1694-7851-2023-2-299-309.
24. The Python SQL Toolkit and Object Relational Mapper [Интернет ресурс]. - URL: <https://www.sqlalchemy.org/> (дата обращения: 22.03.2025).
25. Documentation for Pydantic [Интернет ресурс]. - URL: <https://docs.pydantic.dev/latest/> (дата обращения: 23.03.2025).
26. Репозиторий проекта [Электронный ресурс]. - URL: <https://github.com/moximilian/Diploma> (дата обращения: 02.01.2025).
27. RFC 7231 — Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content / Fielding, R., Reschke, J. [Электронный ресурс]. – URL: <https://clck.ru/3MEj6C> (дата обращения: 06.04.2025).
28. Martin, J. Managing the Data-base Environment. – Englewood Cliffs, N.J. : Prentice-Hall, 2010. – 381 p. – ISBN 0-135-50582-8.
29. Мухортов, В. В. Объектно-ориентированное программирование, реализация и дизайн : метод. пособие / В. В. Мухортов, В. Ю. Рылов. – Новосибирск : ООО «Новософт», 2002. – 108 с.
30. Пилькевич, П. В. Способы улучшения защищённости сервисов, использующих JWT-токены / П. В. Пилькевич // Сб. материалов I Молодежной науч.-практ. Конф., Севастополь, 08 февр. 2023 г. – Севастополь : СевГУ, 2023. – С. 96–97. – URL: <https://clck.ru/3MEj8z> (дата обращения: 01.04.2025).
31. Старанцова, Е. В. Методы хеширования паролей / Е. В. Старанцова // Пути инновационного развития науки и образования в современных условиях : сб. науч. Тр. – Миасс : Изд-во «Аниго», 2023. – С. 387–394. – URL: <https://www.elibrary.ru/item.asp?id=61890426> (дата обращения: 01.04.2025).
32. Веселова, С. В. Цифровая обработка изображений : учеб. пособие / С. В. Веселова, Е. В. Константинова, И. В. Александрова. – СПб. : СПбГИ-

КиТ, 2021. – 283 с. – ISBN 978-5-94760-493-1. – URL: <https://clck.ru/3MEjB4> (дата обращения: 22.03.2025).

33. Дьяков, А. Г. Javascript для создания веб-приложений / А. Г. Дьяков, А. Н. Шмелева // Уральский научный вестник. – 2022. – Т. 10, № 2. – С. 70–74. – URL: <https://clck.ru/3MEjDW> (дата обращения: 02.04.2025).

34. Еремин, М. В. Сравнительный анализ популярных JavaScript фронтенд решений / М. В. Еремин // Тенденции развития науки и образования. – 2022. – № 87-1. – С. 64–68. – DOI 10.18411/trnio-07-2022-14. – URL: <https://www.elibrary.ru/item.asp?id=49278538> (дата обращения: 02.04.2025).

35. Механизмы отрисовки. Документация Vue.js [Электронный ресурс]. – URL: <https://ru.vuejs.org/guide/extras/rendering-mechanism.html> (дата обращения: 02.04.2025).

36. Навигация. Маршрутизация. Документация Vue.js [Электронный ресурс]. – URL: <https://vue-router-ru.netlify.app/guide/> (дата обращения: 02.04.2025).

37. Fetch API. Сетевые запросы [Электронный ресурс]. – URL: <https://learn.javascript.ru/fetch-api> (дата обращения: 12.04.2025).

38. Реактивность. Документация Vue.js [Электронный ресурс]. – URL: <https://clck.ru/3MEjFf> (дата обращения: 02.04.2025).

39. Авторизация через учетную запись Yandex [Электронный ресурс]. – URL: <https://oauth.yandex.ru/client> (дата обращения: 24.04.2025).

40. Авторизация через учетную запись Google [Электронный ресурс]. – URL: <https://console.cloud.google.com/> (дата обращения: 24.04.2025).

41. Что такое Docker? Подробное пособие от создателей [Электронный ресурс]. – URL: <https://docs.docker.com/get-started/docker-overview/> (дата обращения: 02.05.2025).

42. Круглик, Р. И. Использование инструмента Webpack для сборки веб-приложений / Р. И. Круглик // Постулат. – 2020. – № 1(51). – С. 52. – URL: <https://www.elibrary.ru/item.asp?id=42763373> (дата обращения: 12.02.2025).

43. Основная функциональность HTTP сервера Nginx [Электронный ресурс]. – URL: <https://nginx.org/ru/> (дата обращения: 03.05.2025).

44. Механизмы рендера Vue компонентов. Документация Vue.js [Электронный ресурс]. – URL: <https://clck.ru/3MEjJN> (дата обращения: 12.03.2025).

45. Габдуллин, И. И. Сравнение VDS/VPS серверов с виртуальным хо-

стингом и физическим выделенным сервером / И. И. Габдуллин, И. А. Кожевникова // Достижения и приложения современной информатики, математики и физики : материалы IX Всерос. науч.-практ. конф., Нефтекамск, 29 нояб. 2021 г. – Уфа : БашГУ, 2021. – С. 39–42. – URL: <https://clck.ru/3MEjKo> (дата обращения: 04.05.2025).

46. Что такое DNS простыми словами [Электронный ресурс]. - URL: <https://clck.ru/3MEjMo> (дата обращения: 06.05.2025).

47. Обзор зарубежных и российских регистраторов доменных имён [Электронный ресурс]. – URL: <https://habr.com/ru/articles/338206/> (дата обращения: 28.04.2025).

48. Веб приложение для администрирования групповых занятий [Сайт]. - URL: <http://m-syrov.ru:1024/auth/login> (дата обращения: 07.05.2025).

49. Александрова, Е. Г. Свойства качественных тест-кейсов / Е. Г. Александрова, Н. Н. Добрынина // Современные технологии и научно-технический прогресс. – 2024. – № 11. – С. 107–108.

50. Денисенко, А. В. Тестирование веб-приложений: автоматизация против ручного тестирования / А. В. Денисенко // Цифровая трансформация: вчера, сегодня, завтра : Материалы научно-технологической конференции, посвященной юбилею кафедры цифровых и аддитивных технологий СПб-ГУПТД, Санкт-Петербург, 09 октября 2024 года. – Санкт-Петербург: Санкт-Петербургский государственный университет промышленных технологий и дизайна, 2025. – С. 196-199.

51. Архипкин, В. М. Повышение скорости работы информационных систем с использованием Redis / В. М. Архипкин // Информационные технологии в процессе подготовки современного специалиста : межвуз. сб. науч. тр. – Липецк : ЛГПУ им. П. П. Семенова-Тян-Шанского, 2024. – С. 30–35.

ПРИЛОЖЕНИЕ А
Техническое задание

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное Образовательное
учреждение высшего образования МОСКОВСКИЙ
ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ (МОСКОВСКИЙ
ПОЛИТЕХ)

УТВЕРЖДАЮ

УТВЕРЖДАЮ

Руководитель (должность,
наименование предприятия –
заказчика АС

Руководитель (должность,
наименование предприятия –
разработчик АС)

_____/ И. О. Фамилия

_____/ И. О. Фамилия

Дата: «_» _____ 20_ г.

Дата: «_____» _____ 20_ г.

Автоматизированная информационная система
ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ АДМИНИСТРИРОВАНИЯ
ГРУППОВЫХ ЗАНЯТИЙ

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

На 12 листах

Действует с «___» _____ 20_ г.

СОГЛАСОВАНО

Руководитель (должность, наименование
согласующей организации

_____/ И. О. Фамилия

Дата: «_» _____ 20_

СОДЕРЖАНИЕ

1 ОБЩИЕ СВЕДЕНИЯ	82
1.2 Наименование приложения	82
1.3 Наименование организаций	82
1.4 Краткая характеристика области применения	82
1.5 Источники финансирования	82
1.6 Состав используемой нормативно-технической документации	82
2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ	84
2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ	84
2.1 Назначение приложения	84
2.2 Цели создания приложения	84
2.3 Критерии достижения цели	84
3 ХАРАКТЕРИСТИКА ОБЪЕКТОВ АВТОМАТИЗАЦИИ	85
3.1 Объект автоматизации	85
3.2 Существующее нормативно-правовое обеспечение	85
4 ТРЕБОВАНИЯ К АВТОМАТИЗИРОВАННОЙ СИСТЕМЕ	86
4.1 Требования к системе в целом	86
4.2 Требования к функциям системы	87
5 ТРЕБОВАНИЯ К АВТОМАТИЗИРОВАННОЙ СИСТЕМЕ	88
6 ПОРЯДОК РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ	89
7 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ	90
8 ПОДГОТОВКЕ ОБЪЕКТА АВТОМАТИЗАЦИИ К ВВОДУ В ДЕЙ- СТВИЕ	91
9 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ	92
10 ИСТОЧНИКИ РАЗРАБОТКИ	93

1 ОБЩИЕ СВЕДЕНИЯ

1.2 Наименование приложения

1.2.1 Полное наименование приложения

Веб-приложение администрирования групповых занятий.

1.2.2 Краткое наименование приложения

Отсутствует.

1.3 Наименование организаций

1.3.1 Заказчик

Инициатива разработчика.

1.3.2 Разработчик

Сыров Максим Евгеньевич.

1.4 Краткая характеристика области применения

Веб приложение администрирования групповых занятий используется для автоматизированного учета и администрирования занятий, направленных на проведение образовательной деятельности. Основными потребителями являются частные репетиторы, а также небольшие частные образовательные организации. Приложение предлагает обширный функционал учета оплаты, посещаемости, организации занятий, а также мониторинга расписания.

1.5 Источники финансирования

Проект выполняется на безвозмездной основе.

1.6 Состав используемой нормативно-технической документации

При разработке веб-приложения и создании проектно-эксплуатационной документации Исполнитель должен руководствоваться требованиями следующих нормативных документов:

1. Информационная технология. Комплекс стандартов на автоматизированные системы (класс стандартов ГОСТ 34).
2. ГОСТ Р 59853-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Термины и определения».
3. ГОСТ 34.201-2020 Межгосударственный стандарт. «Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем».
4. ГОСТ 19.201 Единая система программной документации.
5. ГОСТ Р 59792-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды испытаний автоматизированных систем».
6. ГОСТ Р 59795-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Требования к содержанию документов».
7. ГОСТ Р 59793-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания».
8. ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ

2.1 Назначение приложения

Основное назначение веб-приложения администрирования групповых занятий это автоматизация процесса проведения образовательной деятельности.

2.2 Цели создания приложения

Основными целями являются

1. Упрощение процесса учета обучающихся перед проведением занятия.
2. Упрощение процесса учета оплаты платных занятий.
3. Автоматизация процесса администрирования занятий обучающихся по группам.
4. Сокращение временных затрат на подготовку занятия преподавателем.

2.3 Критерии достижения цели

Для реализации поставленных целей веб-приложение должно решать следующие задачи:

1. Создание групп с настройкой параметров (макс. количество учеников, стоимость занятий).
2. Составление гибкого расписания: повторяющиеся и разовые занятия. Перенос занятий.
3. Отметка посещаемости с возможностью добавления комментариев.
4. Возможность указать стоимость каждого занятия.
5. Управление контентом: добавление материалов к занятиям и домашнего задания.
6. Уведомление учеников о переносах или отмены занятий.
7. Личный кабинет с расписанием и историей посещений занятий.
8. Вступление в группы преподавателей и просмотр открытых окон.
9. Возможность поиска преподавателей.

3 ХАРАКТЕРИСТИКА ОБЪЕКТОВ АВТОМАТИЗАЦИИ

3.1 Объект автоматизации

Процессы подготовки к занятиям, учета предстоящих занятия, а также успеваемости, посещения и оплаты занятий обучающимися.

3.2 Существующее нормативно-правовое обеспечение

Существующее нормативно-правовое обеспечение составляют федеральные, областные и локальные нормативные правовые акты:

1. ГОСТ Р 51904-2002 программное обеспечение встроенных систем. Общие требования к разработке и документированию.
2. ГОСТ Р 59407-2021 методы и средства обеспечения безопасности. Базовая архитектура защиты персональных данных.

4 ТРЕБОВАНИЯ К АВТОМАТИЗИРОВАННОЙ СИСТЕМЕ

4.1 Требования к системе в целом

4.1.1 Перспективы развития и модернизации системы

1. Расширение функционала: интеграция с платежными системами, добавление мобильного приложения, внедрение аналитики успеваемости.
2. Оптимизацию производительности: масштабирование серверной части, кэширование данных, улучшение отзывчивости интерфейса.
3. Коммерциализацию: запуск пилотного внедрения в локальных образовательных проектах, разработка монетизации (подписка, премиум-функции).
4. Масштабирование: адаптацию платформы для корпоративного обучения и дополнительных рынков.

4.1.2 Требования к численности и квалификации персонала

Для эксплуатации системы достаточно одного администратора, обладающего базовыми знаниями в администрировании веб-сервисов.

4.1.3 Требования к показателям назначения

Система должна обеспечивать возможность одновременной работы 100 пользователей с откликом на операции не более 2 секунд.

4.1.4 Требования к надежности

Система должна сохранять работоспособность при сбоях и обеспечивать восстановление в течение 12 часов. Данные должны автоматически резервироваться каждые 24 часа.

4.1.5 Требования к эргономике и технической эстетике

Интерфейс должен быть выполнен в светлых тонах, обеспечивать интуитивно понятную навигацию и быть адаптированным для работы на ПК, планшетах и смартфонах.

4.1.6 Требования к защите данных от несанкционированного доступа

Система должна поддерживать авторизацию пользователей и защиту от SQL-инъекций.

4.1.7 Требования по сохранности информации при авариях

Все данные системы должны быть защищены регулярным резервным копированием.

4.1.8 Требования по патентной чистоте

Веб-приложение администрирования групповых занятий должно соответствовать требованиям патентной чистоты и исключать нарушение авторских прав.

4.1.9 Требования к стандартизации и унификации

Элементы интерфейса должны быть унифицированы и выполнены в едином стиле, обеспечивая удобство работы пользователей.

4.2 Требования к функциям системы

4.2.1 Подсистема хранения данных

Должна обеспечивать хранение данных о пользователях, созданных ими группах, групповых занятиях и свободных занятиях.

4.2.2 Подсистема авторизации

Авторизация пользователей через логин и пароль с защитой от подбора пароля. Авторизация пользователей через сервис авторизации, предоставленный Yandex и Google.

4.2.3 Базовая подсистема

Предоставление пользователю инструментов для просмотра, создания, редактирования и удаления своих групп обучающихся, создания, просмотра и редактирования информации о созданных занятиях внутри одной группы. Также предоставление инструментов для просмотра и редактирования своих занятий в виде календаря. Для обучающихся также необходимы инструменты для вступления, покидания групп и занятий.

5 ТРЕБОВАНИЯ К АВТОМАТИЗИРОВАННОЙ СИСТЕМЕ

Этапы работ по созданию веб-приложения для администрирования групповых занятий представлены в таблице 5.1.

Таблица 5.1 - Этапы работ по созданию веб-приложения

Этап	Содержание работ	Сроки	Результаты
1. Составление технического задания (ТЗ)	Формирование требований к функциональности и характеристикам системы	7 дней	Готовое техническое задание
2. Проектирование	Разработка макетов интерфейса для подсистем веб-сервиса	5 дней	Макеты интерфейса для всех подсистем
	Создание сценариев работы подсистем веб-сервиса	3 дня	Детализированные сценарии работы системы
	Подготовка дизайн-макета интерфейса для подсистем веб-сервиса	10 дней	Дизайн-макет пользовательского интерфейса
3. Разработка	Разработка клиентской части веб-сервиса	20 дней	Реализованная клиентская часть системы
	Создание серверной части веб-сервиса	30 дней	Реализованная серверная часть системы
4. Тестирование	Проверка соответствия системы требованиям дизайна и функциональности. Тестирование работоспособности без интеграции с внешними системами. Внесение доработок и повторное тестирование для устранения недочетов.	16 дней	Подготовленный к запуску веб-сервис

6 ПОРЯДОК РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

Этапы организации разработки автоматизированной системы для администрирования групповых занятий представлены в Таблице 6.1.

Таблица 6.1 - Этапы организации разработки

Этап	Содержание работ	Сроки	Результаты
1. Разработка серверной части	Проектирование и создание базы данных	5 дней	Созданная база данных
	Реализация модуля для работы с базой данных	10 дней	Реализованный модуль взаимодействия с базой данных
	Разработка API для взаимодействия между клиентом и сервером	12 дней	Реализованный модуль API
2. Разработка клиентской части	Создание концепции пользовательского интерфейса	9 дней	Подготовленные макеты пользовательского интерфейса
	Реализация пользовательского интерфейса	20 дней	Разработанный пользовательский интерфейс
	Разработка API для связи клиентской части с серверной	4 дней	Реализованный модуль взаимодействия клиент-сервер
3. Тестирование	Проверка функциональности серверной части	3 дней	Выявленные ошибки в серверной части
	Исправление ошибок серверной части	5 дней	Устраненные ошибки серверной части
	Проверка функциональности клиентской части	3 дней	Выявленные ошибки в клиентской части
	Исправление ошибок клиентской части	5 дней	Устраненные ошибки клиентской части

7 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ

Виды, содержание, объем и способы проверки подсистемы должны быть описаны в программе и методике испытаний, которые разрабатываются как часть рабочей документации.

8 ПОДГОТОВКЕ ОБЪЕКТА АВТОМАТИЗАЦИИ К ВВОДУ В ДЕЙСТВИЕ

В процессе реализации проекта на объекте автоматизации необходимо выполнить подготовительные работы для ввода системы в эксплуатацию. Для успешного развертывания веб-приложения требуется выполнить следующие шаги:

1. Настроить серверное оборудование для размещения системы.
2. Зарегистрировать и подготовить доменное имя для веб-сервиса.
3. Провести тестовую эксплуатацию системы для проверки ее работоспособности и устранения возможных недостатков.

9 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ

Документация по проекту должна разрабатываться с учетом требований комплекса государственных стандартов:

1. Аналитическая записка (ГОСТ 7.32-2017).
2. Техническое задание (ГОСТ 34.602-2020).
3. Технический проект (ГОСТ 2.120-2013).
4. Пояснительная записка (ГОСТ 2.106-2019).
5. Программа и методика испытаний (ГОСТ 2.106-2019).

10 ИСТОЧНИКИ РАЗРАБОТКИ

Техническое задание и АС разработаны в соответствии со следующими документами:

1. Информационная технология. Комплекс стандартов на автоматизированные системы (класс стандартов ГОСТ 34).
2. ГОСТ Р 59853-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Термины и определения».
3. ГОСТ Р 59792-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды испытаний автоматизированных систем».
4. ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».
5. ГОСТ 19.201 Единая система программной документации. Техническое задание. Требования к содержанию и оформлению
6. ГОСТ Р 59795-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Требования к содержанию документов».
7. ГОСТ 34.201-2020 Межгосударственный стандарт. «Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем».
8. ГОСТ Р 59793-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания».
9. Обоснование и разработка требований к программным системам: учебное пособие / А. А. Бирюкова, А. М. Володина, К. В. Гусев, А. Н. Мионов. — Москва: РТУ МИРЭА, 2022. — 157 с. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/240089> (дата обращения: 06.04.2025). — Режим доступа: для авториз. пользователей.

ПРИЛОЖЕНИЕ Б

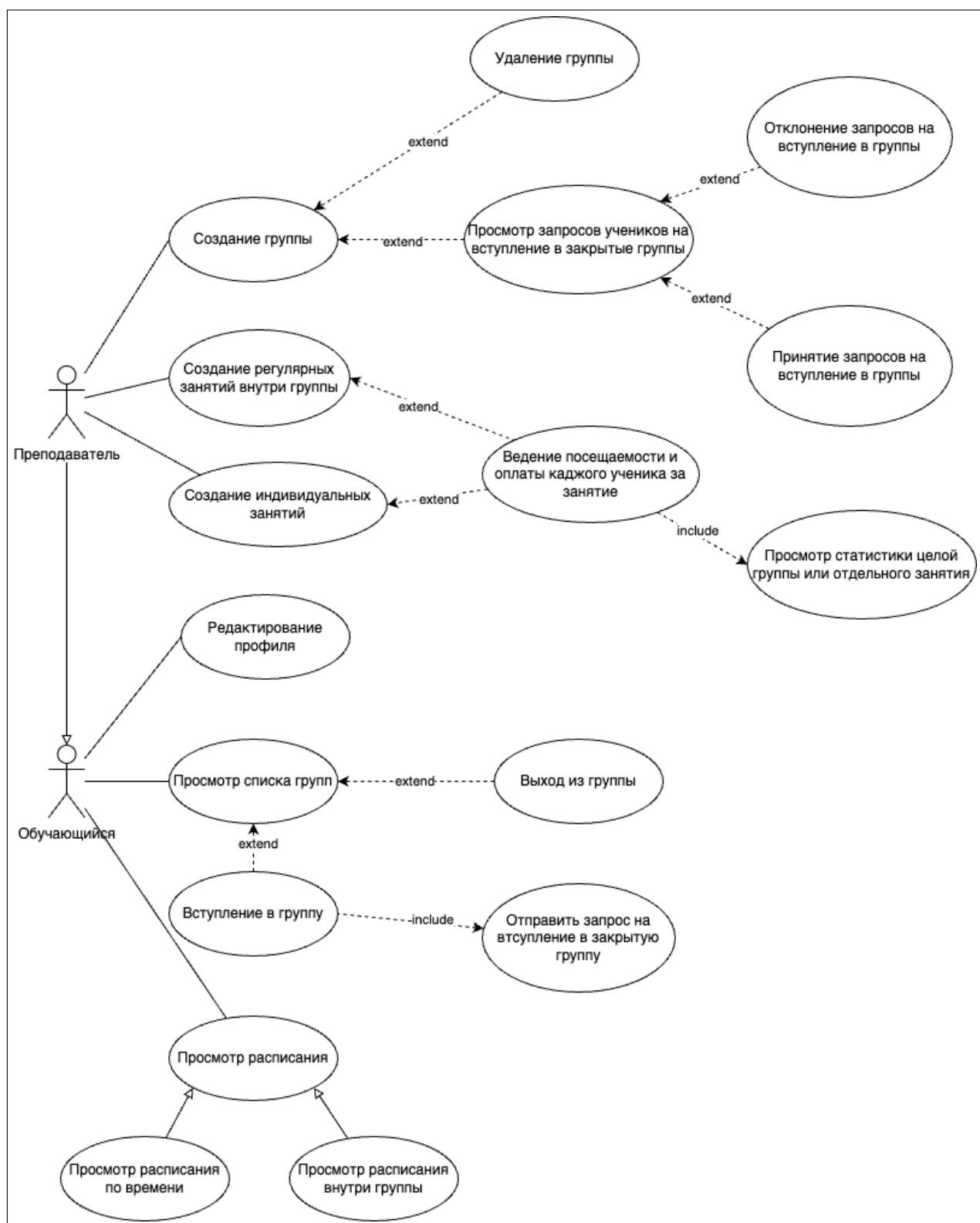


Рисунок Б.1 – UML диаграмма по пользователям

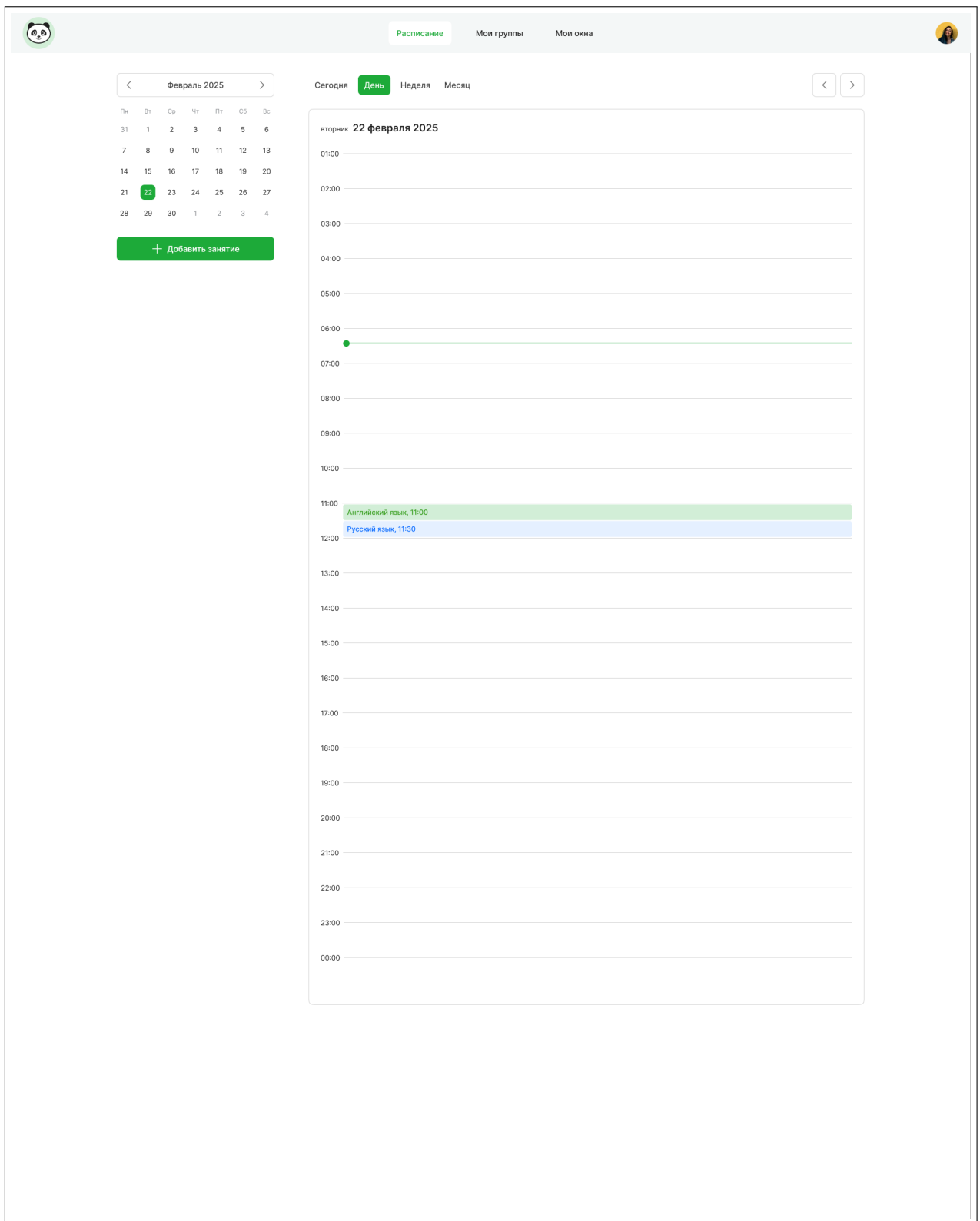


Рисунок Б.2 – Прототип страницы "Расписание". Календарь, день

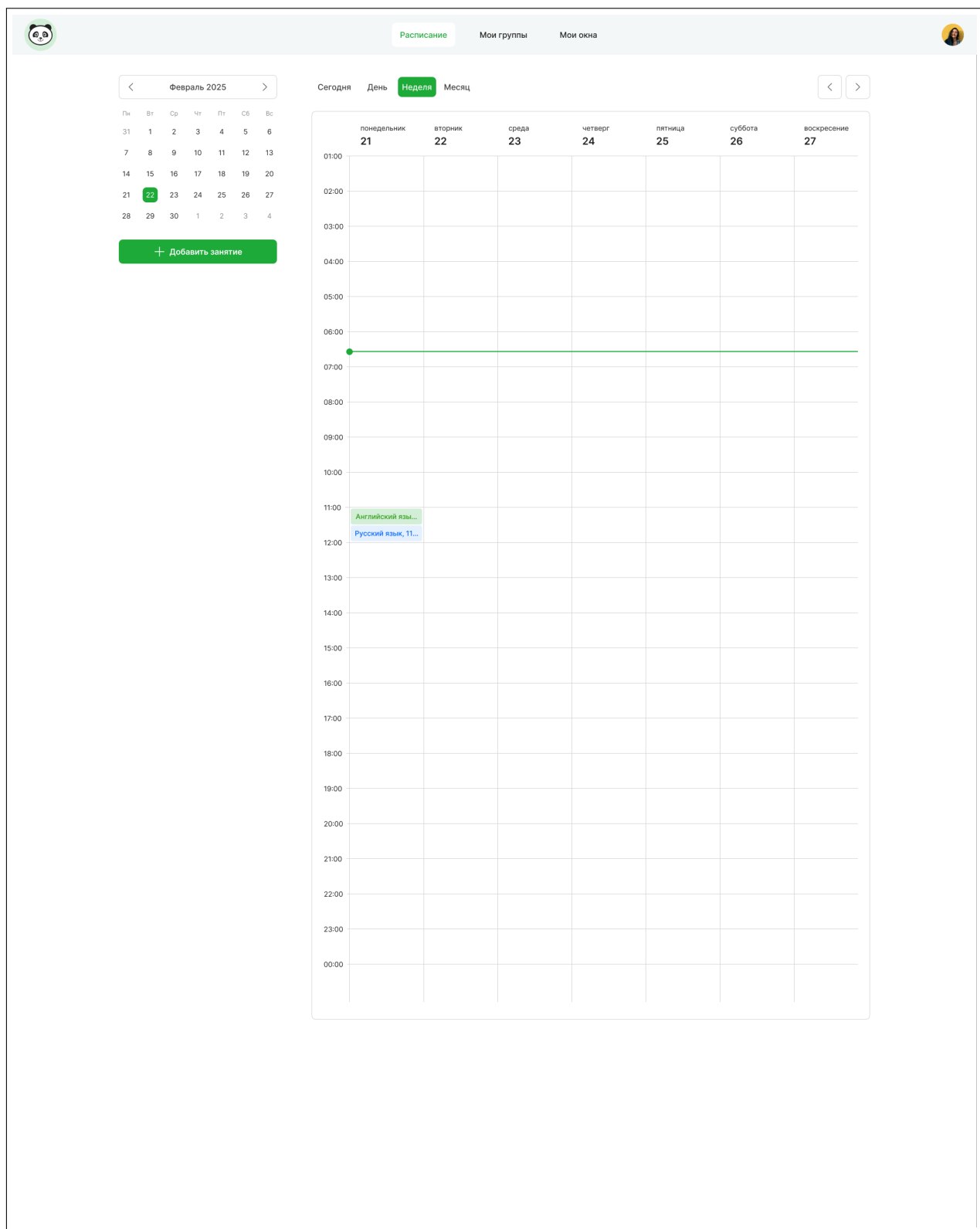


Рисунок Б.3 – Прототип страницы "Расписание". Календарь, неделя

ПРИЛОЖЕНИЕ В

Листинг В.1 — Файл base.py с классом BaseCRUD

```
from sqlite3 import IntegrityError
import models as m
import api.exceptions as exc
from schemas import BaseListResponse
from functools import wraps
import typing as t

class BaseCRUD():
    """BaseCRUD implements base common logic that can be used
    by all other CRUD classes.

    Args:
        db: SQLAlchemy Session instance
        model: SQLAlchemy orm default model
        joined_models: Optional list of models in which filters will work.
    Defaults to [model]
    """

    def __init__(self, db, model, joined_models = []):
        self.db = db
        self.model = model
        self.joined_models = [model] if len(joined_models) == 0 else
joined_models

    def _save_to_db(self, item: t.Dict[str, t.Any]):
        """Create new Item in db and return its new glory.

        Args:
            item: children of BaseModel

        Returns:
            created_item: children of BaseModel
        """
        self.db.add(item)
        self.db.commit()
        self.db.refresh(item)
        return item

    def get_item(self, body, field='id', model=None) -> m.BaseModel:
        """Get one item from specific table by given field key and value.
        Args:
            value (schemas.RequestBodyOne) Request body
```

Продолжение листинга В.1

```
"""
    field (str): Field key of model of which type is searched.
    Defaults to 'id'

    Returns:
        Item in DB that was found in db.
        None if provided field is not found on model.
"""
model = self.model if model is None else model

query = self.db.query(model)
if isinstance(field, str):
    if not hasattr(model, field):
        raise exc.NotFoundError(message='No such field on model',
field=field)
    value = body.get(field)
    if value is None:
        raise exc.NotFoundError(message='Value is null', field=field)
    found_item = query.filter(getattr(model, field) == value).first()
elif isinstance(field, list):
    for field_item in field:
        if not hasattr(model, field_item):
            raise exc.NotFoundError(message='No such field on model',
field=field_item)
        value = body.get(field_item)
        if value is None: raise exc.NotFoundError(message='Value is
null', field=field)
        query = query.filter(getattr(model, field_item) == value)
        found_item = query.first()

if found_item and found_item.get('is_deleted', False):
    raise exc.NotFoundError('Item is deleted')
else:
    return found_item

def get(self, body, field='id', model=None):
    """Get one item from specific table by given field key and value.
    Args:
        value (schemas.RequestBodyOne) Request body
        field (str): Field key of model of which type is searched.
        Defaults to 'id'
    Returns:
        Item in DB that was found in db.
        None if provided field is not found on model.
    return self._make_output([self.get_item(body, field, model))][0]
```

Продолжение листинга В.1

```
def _make_output(self, rows):
    return rows

def _transform_response(self, rows):
    """Transform response into defined format.

    Args:
        rows (list): List of items to wrap

    Returns:
        schemas.BaseListResponse
        .. code-block:: json
        {
            "rows": [],
            "totalCount" : 0
        }
    """
    rows = self._make_output(rows)
    rows = [row.dict() if not isinstance(row, dict) else row for row in
rows]
    return BaseListResponse(rows=rows, totalCount=len(rows))

def _transform_boolean(self, where):
    if isinstance(where, dict) and where.get('value') and where.get('value
') in ['false', 'true']:
        where['value'] = where['value'].lower() in ("yes", "true", "t", "1
")
    return where

def _get_column_from_joined_models(self, column_name):
    for model in self.joined_models:
        if hasattr(model, column_name):
            return getattr(model, column_name)
        if column_name in model.__table__.columns:
            return model.__table__.columns[column_name]
    raise ValueError(f"Column '{column_name}' not found in any of the
provided models")

def list(self, body, model=None):
    """Get one or multiple items in list representation:

    Args:
        body(schemas.RequestBodyList)
        .. code-block:: json
        {
            "filters": {
```

Продолжение листинга В.1

```
"""
        "wheres" : [
            {
                "column":column,
                "condition": condition,
                "value":value
            }
        ],
        "orders": [
            {
                "column": column,
                "desc": boolean
            }
        ],
        "search" : search value @todo,
        "page": integer @todo
        "limit": integer @todo
    }
}
model (model): sqlalchemy model on which to perform queries.
Defaults to None, uses one in defined in constructor

Returns:
    response (schemas.BaseListResponse)
"""
rows = self.get_items(body, model)
return self._transform_response(rows)

def get_items(self, body, model=None):
    model = self.model if model is None else model
    filters = body.get('filters', {})

    wheres = filters.get('wheres', [])
    orders = filters.get('orders', [])

    query = self.db.query(model)
    if hasattr(model, 'is_deleted'):
        query = query.filter(getattr(model, 'is_deleted') == False)
    query, wheres = self._make_custom_query(query, wheres)
    for where in wheres:
        condition = where.get('condition')
        where = self._transform_boolean(where)
        column = self._get_column_from_joined_models(where['column'])
        if condition == 'between':
            query = query.filter(column.between(*where['value']))
        elif condition == '!=':
```

Продолжение листинга B.1

```
        query = query.filter(column != where['value'])
    elif condition == 'in':
        query = query.filter(column.in_(where['value']))
    elif condition == '%':
        query = query.filter(column.contains(where['value']))
    else:
        query = query.filter(column == where['value'])
for order in orders:
    query = query.order_by(getattr(model, order['column']).desc(
        ) if order['desc'] else getattr(model, order['column']).asc())

print(query.statement.compile())

return query.all()

def _make_custom_query(self, query, wheres):
    return query, wheres

def _delete_item(self, body, field, model=None):
    item_to_delete = self.get_item(body, field, model)
    if not item_to_delete:
        raise exc.NotFoundError(field=field)
    self.db.delete(item_to_delete)
    self.db.commit()
    return item_to_delete

def delete(self, body, field='id', model=None):
    """Delete one item from specific table by given field key and value.

    Args:
        body (RequestBodyOne or RequestBodyOnes) Request body
        field (str): Field key of model of which type is searched.
    Defaults to 'id'
        model (model): sqlalchemy model on which to perform queries.
        Defaults to None, uses one in defined in constructor

    Returns:
        Item in DB that was found in db.
        None if provided field is not found on model.
    """
    model = self.model if model is None else model

    if not hasattr(model, field):
        raise exc.NotFoundError(field=field)
    if body.get('ids') and isinstance(body.get('ids'), list):
        items_to_delete = []
        for item in body.get('ids'):
```

Продолжение листинга В.1

```
        item_to_delete = self._delete_item({'id': item}, field, model)
        items_to_delete.append(item_to_delete)
    return items_to_delete
else: return self._delete_item(body, field, model)

def _is_value_empty(self, value) -> bool:
    null_values = ['', 'null', 'None', b'']
    return value in null_values

def _is_immutable_field(self, field) -> bool:
    immutable_fields = ['id', 'login']
    return field in immutable_fields

def update(self, body, model=None):
    """Update one item from one specific table by given fields

    Args:
        body (schemas.RequestBodyOne) Request body
        model (model): sqlalchemy model on which to perform queries.
            Defaults to None, uses one in defined in constructor

    Returns:
        Item in DB that was found in db.
    """
    model = self.model if model is None else model
    item_to_update = self.get_item(body, 'id', model)
    if not item_to_update:
        raise exc.NotFoundError()

    valid_values = {}

    body = body.dict() if hasattr(body, 'dict') else body

    for key, value in body.items():
        if self._is_value_empty(value) or self._is_immutable_field(key):
            continue
        valid_values[key] = value

    self.db.query(model).filter(getattr(model, 'id') ==
    item_to_update.get('id')).update(
        valid_values
    )

    self.db.commit()
    self.db.refresh(item_to_update)

    return item_to_update
```

Продолжение листинга В.1

```
def mark_deleted(self, body, field='id', restore=False, model=None):
    """Mark one item as deleted from specific table by given field key and
    value.

    Args:
        value (schemas.RequestBodyOne) Request body
        field (str): Field key of model of which type is searched.
    Defaults to 'id'
        restore (bool): Flag by which item can be marked or restored.
    Defaults to False.as_integer_ratio
        model (model): sqlalchemy model on which to perform queries.
        Defaults to None, uses one in defined in constructor

    Returns:
        Item in DB that was found in db.
        None if provided field is not found on model.
    """
    model = self.model if model is None else model
    if not hasattr(model, 'is_deleted'):
        print(f'{model} do not have `is_deleted` property')
        raise exc.InternalError()

    item_to_delete = self.get_item(body, field, model)
    if not item_to_delete:
        raise exc.NotFoundError(field=field)

    # setattr(item_to_delete, 'is_deleted', (not restore))
    self.db.query(model).filter(getattr(model, field) == item_to_delete.
get(
    field)).update({'is_deleted': (not restore)})
    self.db.commit()
    self.db.refresh(item_to_delete)

    return item_to_delete

def insert(self, request_body, model=None) -> dict:
    model = self.model if model is None else model
    if not isinstance(request_body, list):
        request_body = [request_body]
    inserted_items = [
        model(
            **{
                k: v for k, v in (item.items() if isinstance(item, dict)
else item.model_dump().items()) if k in model.__table__.columns
            }
        ) for item in request_body
    ]
```

Окончание листинга B.1

```
        created_items = []
    for item in inserted_items:
        created_items.append(self._save_to_db(item))
    return created_items
```

Листинг B.2 — Файл auth.py с классом Authorisation

```
"""
Logic to work with DB
"""

from datetime import datetime, timedelta, timezone
from sqlalchemy.orm import Session
from sqlalchemy import and_
import models as m
from schemas import UserCreate, UserLogin, UserOut, Token
from fastapi import Depends
from fastapi.security import OAuth2PasswordBearer
from crud.base import BaseCRUD
import api.exceptions as exc
from database import get_db
from jose import JWTError, jwt
from env_constants import SECRET_KEY, ACCESS_TOKEN_EXPIRE_MINUTES, BASE_URL
import typing as t

import bcrypt

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl=f'/{BASE_URL}/auth/login', auto_error=False)
ALGORITHM = 'HS256'

async def authorised_user(token: str = Depends(oauth2_scheme), db: Session =
    Depends(get_db)) -> UserOut:
    """Dependency by which access is guaranteed only to authorised users.
    Token should be placed in Authorisation Header of any request that
    requires authorised access.

    Example of Header placement: `Authorization: 'Bearer <your JWT token>'`

    Args:
        `token` (str): JWT access token from headers

    Returns:
        Authorised user

    Raises:
        AuthorisationError: If provided token is mismatched or provided login
```


Продолжение листинга В.2

```
"""
    does not exist
"""
if Authorisation(db)._is_token_revoked(token):
    raise exc.AuthorisationError('Token is expired. Log in again')
try:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    username = payload.get('sub')
    if username is None:
        raise exc.AuthorisationError('Could not validate credentials')
except (JWTError, AttributeError):
    raise exc.AuthorisationError('Provided token is invalid')
user = Authorisation(db)._get_user_by_login(username)
if user is None:
    raise exc.AuthorisationError('Could not validate credentials')
return user

class Authorisation(BaseCRUD):
    def __init__(self, db: Session):
        super().__init__(db, m.User)

    def register(self, body: UserCreate) -> UserOut:
        """Register user.

        Args:
            `body` (UserCreate): User data for creation

        Returns:
            New registered user

        Raises:
            """
            exc.ValidationError

        login = body.get('login')
        existing_user = self._get_user_by_login(login)
        if (existing_user):
            raise exc.ValidationError(
                message='Username already registered', field='login')

        password = body.get('password')
        password_confirm = body.get('password_confirm')
        if (password != password_confirm):
            raise exc.ValidationError(
                message='Passwords do not match', field='password_confirm')
```

Продолжение листинга В.2

```
        hashed_password = self._hash_password(password)

        new_body = body
        new_body.password = hashed_password
        return super().insert(new_body)[0]

def _hash_password(self, password: t.Optional[str]) -> str:
    """Hash password with bcrypt module.

    Args:
        password (str): plain password from request.

    Returns:
        hashed_password (str): hashed password.
    """
    if password is None:
        return ''
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).
decode('utf-8')

def login(self, auth_creds: UserLogin) -> Token:
    """Authorise user. Create JWT token for future access.

    Args:
        `auth_creds` (UserLogin): Credentials for authorisation

    Returns:
        Token for future access

    Raises:
        ValidationError if db does not have user with provided login
    """
    user = self._verify_token(
        auth_creds.get('login'), auth_creds.get('password')
    )
    if not user:
        raise exc.ValidationError(field=['login', 'password'])

    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = self._create_access_token(
        data={'sub': user.login}, expires_delta=access_token_expires
    )
    return {'access_token': access_token, 'token_type': 'bearer',
            'user_id': user.get('id')}

def _verify_token(self, username: str, password: str) -> t.Union[UserOut,
bool]:
```

Продолжение листинга В.2

```
        """Verify if user exists and given password match one in db.

    Args:
        `username` (str): login for auth.
        `password` (str): plain password for auth check.

    Returns:
        False if there is no such user or passwords do not match. \n
        User if one exists.
    """
    user = self._get_user_by_login(username)
    if not user or not self._verify_password(password, user.get('password'
    )):
        return False
    return user

    def _verify_password(self, plain_password: str, hashed_password: str) ->
    bool:
        """Verify plain password's hash matches one in db.

    Args:
        plain_password (str): plain password from request
        hashed_password (str): hash of plain password stored in db

    Returns:
        bool Result of match
    """
        return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.
    encode('utf-8'))
    def _get_user_by_login(self, login: str) -> t.Optional[t.Union[UserOut,
    None]]:
        """Get user by login.

    Args:
        login (str): Login by which search will be run.

    Returns:
        m.User if one exists with provided login else None.
    """
        return self.db.query(m.User).where(
            and_(m.User.login == login, m.User.is_deleted == False)).first()

    def _create_access_token(self, data, expires_delta: timedelta):
        """Create and encode JWT token for future access.

    Args:

        data: login by which jwt check authorisation.
```

Окончание листинга В.2

```
    """
    expires_delta: duration of time by which token is valid.
    Returns:
        Encoded JWT token
    """
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def logout(self, token: Token) -> dict:
    """Logout from current session

    Args:
        token (Token): Token to be restricted
    """
    revoked_token = m.RevokedToken(token=token.access_token)
    self.db.add(revoked_token)
    self.db.commit()
    return {"message": "Successfully logged out"}

def _is_token_revoked(self, access_token: str) -> bool:
    """Checks if token is revoked.

    Args:
        access_token (str): Token access_token

    Returns:
        bool if token is revoked or not
    """
    return self.db.query(m.RevokedToken).filter(m.RevokedToken.token ==
access_token).first() is not None
```

Листинг В.3 — Генерация маршрутов внутри приложения

```
const generateRoutes = sections => {
  const routes = []
  sections.forEach(section => {
    const defaultComponentName = `${capitalize(section.name)}Page`
    const defaultComponent =
importComponent(`${section.name}/${defaultComponentName}`)
```

Продолжение листинга В.3

```
    if (!section?.actions?.length) {
      routes.push({
        path: `/${section.name}`,
        name: `${section.name}`,
        component: defaultComponent,
      })
    } else {
      section.actions.forEach(action => {
        const component = importComponent(
          `${section.name}/${capitalize(section.name)}${capitalize(
            action
          )}Page`
        )
        console.log(`${section.name}/${capitalize(section.name)}${
          capitalize(action)
        }Page`)
        let pathAction = `/:action(${action})`
        const path =
          section.useId && !['list', 'new'].includes(action)
            ? `/${section.name}/${pathAction}/:id`
            : `/${section.name}/${pathAction}`
        routes.push({
          path,
          name: `${section.name}-${action}`,
          component,
        })
      })
    }
  })
  return routes
}
```

Листинг В.4 — Процесс инициализации авторизации YandexOAuth

```
async initYandexAuth() {
  /* eslint-disable no-undef */
  await this.loadBtnScript(
    'https://yastatic.net/s3/passport-sdk/autofill/v1/sdk-suggest-with-
    polyfills-latest.js'
  )
  try {
    const { handler } = await YaAuthSuggest.init(
      /* eslint-enable no-undef */
      {
        client_id: YANDEX_CLIENT_ID,
        response_type: 'token',
        redirect_uri: `http://${window.location.hostname}:1024/auth/`
      }
    )
  }
}
```

Продолжение листинга В.4

```
        yandex-callback`,
        display: 'page',
      },
      null,
      {
        display: 'page',
        view: 'button',
        parentId: 'auth-bottom',
        buttonView: 'main',
        buttonTheme: 'light',
        buttonSize: 'm',
        buttonBorderRadius: 8,
      }
    )
    await handler()
  } catch (error) {
    console.error('Yandex auth error:', error)
  }
},
created() {
  this.initYandexAuth()
}
```

Листинг В.5 — Процесс авторизации через Google

```
async initGoogleAuth() {
  await this.loadBtnScript('https://accounts.google.com/gsi/client?hl=ru')
  try {
    /* eslint-disable no-undef */
    await google.accounts.id.initialize({
      client_id: GOOGLE_CLIENT_ID,
      callback: ({ credential }) => {
        const userData = jwtDecode(credential)
        const password = this.transformPassword(
          `${userData.sub}-${userData.email.ucFirst()}`
        )
        this.$api.auth.register(
          {
            name: userData.given_name,
          },
          () => {
            this.authorize({
              login: userData.email,
              password: password,
            })
          }
        )
      }
    })
  }
}
```

Продолжение листинга В.5

```
        }
    )
    },
    response_type: 'token',
    scope: 'https://www.googleapis.com/auth/userinfo.profile
https://www.googleapis.com/auth/userinfo.email openid',
    ux_mode: 'popup',
  })
  google.accounts.id.renderButton(document.getElementById('auth-
bottom2'), {
    theme: 'outline',
    size: 'large',
    locale: 'ru',
  })
  google.accounts.id.prompt() // also display the One Tap dialog
  /* eslint-enable no-undef */
} catch (error) {
  console.error('Google auth error:', error)
}
},
created() {
  // other code
  this.initGoogleAuth()
  // other code
}
```